

Scloud⁺

Algorithm Specifications and Supporting Documentation

Anyu Wang

Institute for Advanced Study, Tsinghua University, Beijing, China
Zhongguancun Laboratory, Beijing, China

Zhongxiang Zheng

School of Computer and Cyber Sciences, Communication University of China, Beijing, China

Chunhuan Zhao

Shield Lab, Huawei Technologies, Beijing, China

Zhiyuan Qiu

Shandong Institute of Blockchain, Jinan, China

Yi Tang

Shield Lab, Huawei Technologies, Beijing, China

Changchun Mu

Digital Currency Institute, the Peoples Bank of China

Contact: Anyu Wang anyuwang@tsinghua.edu.cn

April 30, 2026

Table of Contents

1	Algorithm Specification	3
1.1	Key Algorithmic Features	3
1.1.1	Unstructured LWE Design.	3
1.1.2	Lattice Coding.	3
1.1.3	Secret and Noise Distribution.	3
1.1.4	Multi-Platform Implementation.	4
1.1.5	Performance.	4
1.2	Notations	5
1.3	Subroutines.	5
1.3.1	Symmetric Primitives.	5
1.3.2	Random Matrix Generation.	5
1.3.3	Sampling from the Bernoulli-Difference Distribution.	5
1.3.4	Packing and Unpacking.	6
1.3.5	The <code>MsgEnc</code> and <code>MsgDec</code> Functions.	7
1.4	Specification of <code>Scloud⁺.PKE</code>	9
1.5	Specification of <code>Scloud⁺.KEM</code>	10
1.6	Parameter Sets of <code>Scloud⁺</code>	11
1.6.1	Key and Ciphertext Sizes.	12
2	Performance Analysis	12
2.1	Instantiations for the Symmetric Primitives	12
2.2	Matrix Generation and Sampling	13
2.3	Implementation Results	14
2.3.1	AES-based Results.	14
2.3.2	SHAKE-based Results.	14
2.3.3	SM3-based Results.	15
3	Lattice Coding and DFR Analysis.	16
3.1	Lattice Codes and Barnes-Wall Lattices	16
3.1.1	Lattice Codes.	16
3.1.2	Barnes-Wall Lattices.	17
3.1.3	Lattice Coding Based on Barnes-Wall Lattices.	18
3.1.4	Decryption Failure Rate Analysis.	19
4	Security Proofs	20
4.1	Cryptographic Definitions	20
4.2	Security of the PKE Scheme	21
4.3	Security of the KEM Scheme	22
5	Security Against Known Attacks	23
5.1	Lattice Reduction	23
5.2	Attacks for LWE.	23
5.2.1	Primal Attack.	23
5.2.2	Dual Attack.	24
5.2.3	Concrete hardness of our parameter.	24
A	Message Encoding and Decoding for <code>RBW_d</code>	26
B	Integer Versions of the <code>MsgDec</code> and BDD Functions.	28
C	Reference Implementation Results	29
C.1	AES-based Results (Reference Implementation).	29
C.2	SHAKE-based Results (Reference Implementation).	30
C.3	SM3-based Results (Reference Implementation).	30

1 Algorithm Specification

This document presents Scloud^+ , an IND-CCA2-secure key-encapsulation mechanism (KEM) based on the Learning with Errors (LWE) problem. Scloud^+ was first described in [1] and can be viewed as a successor to the Scloud scheme [2] from the National Cryptographic Algorithm Design Competition. Scloud^+ improves upon Scloud by refining the lattice coding and employing a ternary secret, which significantly reduces key and ciphertext sizes as well as improves running speed. This document further tweaks the design of [1] (referred to as Scloud^+ -SSR in this document) in the choice of lattice coding and secret/noise distribution, and provides security parameters beyond 256 bits (specifically 384 and 512 bits).

The construction of Scloud^+ follows a two-stage approach: we first introduce an IND-CPA-secure public-key encryption scheme (referred to as $\text{Scloud}^+.\text{PKE}$). We then apply the Fujisaki-Okamoto transformation with implicit rejection [3] to build the IND-CCA2-secure KEM (referred to as $\text{Scloud}^+.\text{KEM}$).

This document is organized as follows: Section 1 presents the algorithm specifications of Scloud^+ . Section 2 provides implementation considerations and performance analysis. Section 3 discusses the theory of lattice coding in Scloud^+ and provides an analysis of the decryption failure rate. Sections 4 and 5 give the security proofs and a concrete security analysis under known attacks, respectively.

1.1 Key Algorithmic Features

1.1.1 Unstructured LWE Design. LWE-based schemes can be broadly divided into two categories depending on whether they introduce algebraic structure into the LWE problem. Schemes in the first category (*unstructured-LWE*), such as FrodoKEM [4], base their security purely on the hardness of the LWE problem. Schemes in the second category (*structured-LWE*) are built on algebraic variants such as Ring-LWE [5,6] and Module-LWE [7], and include CRYSTALS-Kyber [8], Saber [9], LAC [10], and Aegis [11].

Introducing algebraic structure enables more compact schemes with better computation and communication efficiency. However, it also complicates the security reduction to random lattice problems (approximate SVP, SIVP). On the other hand, structured-LWE variants reduce to algebraically structured problems such as Ideal-SVP and Module-SVP, whose quantum hardness remains debated. In fact, recent results have demonstrated polynomial-time quantum algorithms for approximate Ideal-SVP in certain families of number fields [12,13,14,15], whereas no comparable attack on unstructured lattice problems is known.

Consequently, unstructured-LWE schemes like FrodoKEM and Scloud^+ are regarded as conservative choices for high-security and long-term confidentiality applications [16,17], though their efficiency lags behind structured schemes. Additionally, Scloud^+ tries to narrow this efficiency gap through the combined use of lattice coding and a carefully designed secret distribution, as described below.

1.1.2 Lattice Coding. Scloud^+ designs a robust error correction method based on Barnes-Wall lattice codes, ensuring smaller parameter sizes while maintaining the appropriate decryption failure probability. While the use of lattice coding is common in LWE-based constructions, previous schemes often involve lattice codes with dimensions 4 to 16. Although larger-dimensional lattice codes generally offer better signal-to-noise ratios and thus stronger error correction capabilities, they require specially designed labeling and delabeling processes to efficiently map the message to the lattice code or vice versa, which poses a challenge for high-dimensional lattice codes. For example, the 24-dimensional Leech lattice-based PKE proposed in [18] suffers from a lack of a labeling technique, making it impractical [19]. Scloud^+ overcomes this by designing efficient labeling and delabeling algorithms for Barnes-Wall lattice codes, along with an efficient bounded distance decoding (BDD) algorithm, enabling the use of up to 128-dimensional lattice codes for error correction without compromising the scheme's performance.

1.1.3 Secret and Noise Distribution. Scloud^+ employs the Bernoulli-Difference (BD) distribution for both the secret and noise matrices. We observe that in unstructured-LWE-based

schemes, two of the most time-consuming operations are the generation of the matrix $\mathbf{A}$ and the matrix-vector multiplication, i.e., the computation of $\mathbf{AS}$. Adopting the BD distribution—which takes values in $\{-1, 0, 1\}$ —in Scloud^+ improves noise control during decryption, enabling the use of a smaller ciphertext modulus while ensuring correct decryption. This provides an opportunity to reduce matrix sizes while maintaining the same security level, thereby facilitating faster matrix sampling and more efficient matrix-vector multiplication. Moreover, the BD distribution includes a tunable parameter ρ , allowing flexible balancing between noise variance and sparsity across different security levels.

1.1.4 Multi-Platform Implementation. Scloud^+ provides three complete families of symmetric primitive instantiations—AES, SHAKE, and SM3—demonstrating the algorithm’s flexibility and adaptability across different platforms and compliance requirements. For public matrix generation, the AES family uses AES-128 in CTR mode, the SHAKE family uses SHAKE-128, and the SM3 family uses the SM3 pseudo-XOF. All cryptographic primitives are domain-separated via fixed label bytes to maintain clean random-oracle abstraction. Furthermore, the implementation is optimized for multiple architectures, including x86_64 (Intel Xeon Gold 6248R), Linux AArch64 (HiSilicon Kunpeng 920), and macOS arm64 (Apple M3 Max), with both optimized and reference implementations provided for each platform. The optimized implementations leverage platform-specific SIMD capabilities and compiler optimizations, while the reference implementations ensure portability and correctness verification across platforms.

1.1.5 Performance. Scloud^+ achieves substantial improvements in both computation and communication efficiency compared to FrodoKEM, the most widely recognized unstructured-LWE-based KEM. Table 1 summarizes the key performance (measured on an Intel Xeon Gold 6248R at 3.00 GHz using the AES-optimized implementation) and size comparisons.

Table 1. Performance and size comparison between $\text{Scloud}^+.\text{KEM}$ and FrodoKEM (x86_64, AES-based).

Scheme	KeyGen (10^3 cycles)	Encaps (10^3 cycles)	Decaps (10^3 cycles)	PK + CT (bytes)
$\text{Scloud}^+.\text{KEM-128}$	791.1	838.5	752.2	6096 + 6160
FrodoKEM-640	1050.0	1455.4	1386.7	9616 + 9720
Ratio	75.3%	57.6%	54.2%	63.4% + 63.4%
$\text{Scloud}^+.\text{KEM-192}$	1561.2	1715.8	1658.7	11456 + 12645
FrodoKEM-976	2057.7	2701.1	2590.7	15632 + 15744
Ratio	75.9%	63.5%	64.0%	73.3% + 80.3%
$\text{Scloud}^+.\text{KEM-256}$	3510.9	3828.6	3719.8	16296 + 17925
FrodoKEM-1344	3648.7	4749.6	4631.3	21520 + 21632
Ratio	96.2%	80.6%	80.3%	75.7% + 82.9%

As shown in Table 1, $\text{Scloud}^+.\text{KEM-128}$ achieves a public key size of 6,096 bytes (63.4% of FrodoKEM-640) and a ciphertext size of 6,160 bytes (63.4% of FrodoKEM-640). For computational performance, $\text{Scloud}^+.\text{KEM-128}$ reduces KeyGen to 75.3%, Encaps to 57.6%, and Decaps to 54.2% of the corresponding FrodoKEM-640 operations, with the combined Encaps+Decaps time reduced to approximately 56.0%. Similar advantages are observed at higher security levels: $\text{Scloud}^+.\text{KEM-192}$ achieves a combined Encaps+Decaps time that is only 63.8% of FrodoKEM-976, and $\text{Scloud}^+.\text{KEM-256}$ achieves 80.5% of FrodoKEM-1344. Note that while FrodoKEM-1344 targets 256-bit security, $\text{Scloud}^+.\text{KEM-256}$ uses larger matrices (1184, 1184) compared to (1088, 1088) in FrodoKEM-1344, which partially accounts for the reduced margin at this level; nevertheless, $\text{Scloud}^+.\text{KEM-256}$ still provides a more than 24% improvement in public key size and a 17% improvement in ciphertext size over FrodoKEM-1344.

1.2 Notations

- Vectors are denoted by bold lower-case letters, such as $\mathbf{v}$, while matrices are represented by bold upper-case letters, such as $\mathbf{A}$.
- The Hamming weight of a vector $\mathbf{v}$ is denoted as $w_H(\mathbf{v})$. For any integer $0 \leq n < 2^k$, we define $\text{Bit}(n, k) = (b_0, \dots, b_{k-1}) \in \{0, 1\}^k$ as the base-2 expansion of n , such that $n = \sum_{i=0}^{k-1} b_i \cdot 2^i$. The Hamming weight of the vector $\text{Bit}(n, k)$ is denoted by $w_H(n)$. Note that $w_H(n)$ is well defined as the total weight is independent of the choice of k , for any $k > \log_2(n)$ that is large enough and valid.
- The inner product of vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ is denoted as $\langle \mathbf{u}, \mathbf{v} \rangle$, and the Euclidean norm of a vector $\mathbf{v}$ is denoted as $\|\mathbf{v}\| = \sqrt{\langle \mathbf{v}, \mathbf{v} \rangle}$. The distance between a vector $\mathbf{v}$ and a discrete set $\mathcal{S} \subseteq \mathbb{R}^n$ is defined by $\text{dist}(\mathbf{v}, \mathcal{S}) := \min_{\mathbf{t} \in \mathcal{S}} \|\mathbf{t} - \mathbf{v}\|$.
- For any real number $x \in \mathbb{R}$, let $\lfloor x \rfloor$ denote the largest integer less than or equal to x , and let $\lceil x \rceil := \lfloor x + 1/2 \rfloor$ denote the integer nearest to x , where half-integers are rounded upward. When rounding half-integers downward is required, we write $\lfloor x - 1/2 \rfloor$. We extend the notation to complex numbers by $\lfloor a + bi \rfloor := \lfloor a \rfloor + \lfloor b \rfloor i$. Additionally, $\lfloor x \rfloor_{\text{odd}}$ denotes the closest integer to x , where any half-integer $n + 1/2$ is rounded to the closest odd integer. For integers n and $q > 0$, we denote by $[n]_q$ the unique integer such that $q \mid (n - [n]_q)$ and $0 \leq [n]_q < q$. These notations extend to vectors by applying the operations component-wise.
- Sampling random variable x from a distribution χ is denoted by $x \leftarrow \chi$. The uniform discrete distribution over a finite set $\mathcal{S}$ is denoted by $U(\mathcal{S})$.

1.3 Subroutines

This subsection presents the subroutines used in Scloud^+ , including symmetric cryptographic primitives, random matrix generation, distribution and sampling, packing and unpacking, and message encoding and decoding functions.

1.3.1 Symmetric Primitives. Scloud^+ employs the following symmetric primitives:

- A pseudorandom expansion function $\text{PRF} : \{0, 1\}^* \rightarrow \{0, 1\}^*$, used to derive $\text{seed}_{\mathbf{A}}$, sampling seeds, and encryption coins.
- A public-matrix extendable-output function $\text{XOF} : \{0, 1\}^{128} \times \mathbb{Z} \rightarrow \{0, 1\}^*$.
- Two hash/expansion functions: $\mathbf{H} : \{0, 1\}^* \rightarrow \{0, 1\}^h$ and $\mathbf{G} : \{0, 1\}^* \rightarrow \{0, 1\}^{2h}$.
- A key-derivation function $\text{KDF} : \{0, 1\}^* \rightarrow \{0, 1\}^{l_{\text{ss}}}$.

Here, h denotes the level-dependent hash-material length, and l_{ss} denotes the shared secret length, which are specified in [Section 1.6](#).

1.3.2 Random Matrix Generation. Scloud^+ utilizes a function gen to generate random matrices that are statistically indistinguishable from uniformly random matrices over $\mathbb{Z}_q$. Specifically, given a 128-bit random seed $\text{seed}_{\mathbf{A}}$, gen first invokes an XOF function to produce a random bit string, which is then assembled into an $m \times n$ matrix over $\mathbb{Z}_q$. The exact process of gen depends on the choice of XOF (e.g., AES-128, SHAKE, or SM3), which will be discussed in [Section 2](#).

1.3.3 Sampling from the Bernoulli-Difference Distribution. Scloud^+ involves the use of the Bernoulli-Difference Distribution which is defined as follows.

Definition 1 (Bernoulli-Difference Distribution). *The Bernoulli-Difference distribution with parameter ρ is defined as the distribution of the difference $X - Y$, where X and Y are independent and identically distributed according to $\text{Bernoulli}(\rho)$. Here $\text{Bernoulli}(\rho)$ denotes the Bernoulli distribution with parameter ρ , i.e., $\Pr[X = 1] = \Pr[Y = 1] = \rho$ and $\Pr[X = 0] = \Pr[Y = 0] = 1 - \rho$.*

If Z follows the Bernoulli-Difference distribution with parameter ρ , then Z takes values in $\{-1, 0, 1\}$ with probabilities

$$\Pr[Z = 1] = \Pr[Z = -1] = \rho(1 - \rho), \quad \Pr[Z = 0] = 1 - 2\rho(1 - \rho),$$

and its variance is $\text{Var}(Z) = 2\rho(1 - \rho)$. In particular, the Bernoulli-Difference distribution with parameter $\rho = \frac{1}{2}$ coincides with the centered binomial distribution of variance $\frac{1}{2}$.

In Scloud^+ , we generate matrices whose entries are sampled independently from the Bernoulli-Difference distribution with parameter ρ . The function that performs this, denoted by BD , takes as input a random coin string $\mathbf{r} \in \{0, 1\}^{512}$ along with parameters (m, n) and ρ , and outputs a matrix $\mathbf{E}$ with each entry drawn from the Bernoulli-Difference distribution with parameter ρ . Details of BD are provided in [Algorithm 1](#). We note that BD is defined only for distribution parameters of the form $\rho = 1/p$ such that p is a positive integer, which is precisely the case of interest in Scloud^+ .

Algorithm 1 The BD Function

Input: Random seed $r \in \{0, 1\}^{512}$

Input: Positive integers m, n

Input: Distribution parameter $\rho = 1/p$ where p is a positive integer

Output: A matrix $\mathbf{E} \in \{-1, 0, 1\}^{m \times n}$

```

1: Generate a pseudorandom byte stream  $b_0, b_1, b_2, \dots \leftarrow \text{PRF}(r)$ 
2: Let  $t \leftarrow \lceil \log_2 p \rceil$  ▷ bits needed per Bernoulli trial
3:  $\ell \leftarrow 0$  ▷ bit position in the stream
4: for  $i = 1$  to  $m$  do
5:   for  $j = 1$  to  $n$  do
6:     repeat
7:        $a \leftarrow \sum_{k=0}^{t-1} b_{\ell+k} \cdot 2^k$  ▷ read  $t$  bits as integer
8:        $\ell \leftarrow \ell + t$ 
9:     until  $a < p$  ▷ rejection sampling: accept only if  $a \in [0, p-1]$ 
10:     $x \leftarrow (a = 0)$  ▷  $x \sim \text{Bernoulli}(\rho)$ 
11:    repeat
12:       $b \leftarrow \sum_{k=0}^{t-1} b_{\ell+k} \cdot 2^k$ 
13:       $\ell \leftarrow \ell + t$ 
14:    until  $b < p$ 
15:     $y \leftarrow (b = 0)$  ▷  $y \sim \text{Bernoulli}(\rho)$ 
16:     $e_{i,j} \leftarrow x - y$ 
17:  end for
18: end for
19: return Matrix  $\mathbf{E}$  with entries  $e_{i,j}$ 
```

For each matrix entry, [Algorithm 1](#) draws two independent uniform integers $a, b \in \{0, \dots, p-1\}$ via rejection sampling and outputs $(a = 0) - (b = 0)$, where each $(\cdot = 0)$ evaluates to 1 if the equality holds and 0 otherwise. Thus each entry follows the Bernoulli-Difference distribution with parameter p . When p is a power of two (i.e., $p = 2, 4$ in our parameter sets), we have $t = \log_2 p$, so every t -bit sample automatically satisfies $a < p$. The rejection loop is therefore vacuous and can be omitted. For non-power-of-two values ($p = 6, 12$), rejection sampling is required.

1.3.4 Packing and Unpacking. Scloud^+ uses a packing function Pack and an unpacking function Unpack to convert integer matrices to bit strings and vice versa. Specifically, let $\mathbf{C} \in \mathbb{Z}_q^{m \times n}$ have entries $c_{i,j}$ for $1 \leq i \leq m$, $1 \leq j \leq n$, where $q = 2^t$ is a power of 2. Then $\text{Pack}(\mathbf{C})$ returns the bit string

$$\text{Bit}(c_{1,1}, t) \parallel \text{Bit}(c_{1,2}, t) \parallel \dots \parallel \text{Bit}(c_{1,n}, t) \parallel \text{Bit}(c_{2,1}, t) \parallel \dots \parallel \text{Bit}(c_{m,n}, t) \in \{0, 1\}^{m \cdot n \cdot t}.$$

For our parameters, we have that $m \cdot n \cdot t$ is always a multiple of 8, so the result is always byte-aligned. The function Unpack is the inverse of Pack .

Secret Key Packing. The secret matrix $\mathbf{S} \in \{-1, 0, 1\}^{n \times \bar{n}}$ is encoded using a compact 2-bit-per-coefficient packing. Each ternary coefficient is represented directly by taking the lowest two bits: -1 as 11_2 , 0 as 00_2 , and 1 as 01_2 , and four coefficients are packed into one byte. Since $2n\bar{n}$ is always a multiple of 8 in our parameter sets, the packed secret key $\text{Packsk}(\mathbf{S})$ is byte-aligned without padding.

For **Unpacksk**, let $x \in \{11_2, 00_2, 01_2\}$ be the 2-bit stored value. Setting $g \leftarrow x \gg 1$ extracts the sign bit, and the original coefficient is recovered as $s = x|((-g) \& 11 \dots 1100_2)$, which yields -1 , 0 , or 1 , respectively.

1.3.5 The MsgEnc and MsgDec Functions. Scloud⁺ employs lattice coding to encode binary messages into integer matrices. In particular, our construction uses the Barnes-Wall lattices BW_d and their rotations RBW_d , where $d = 2^k$ is a power of two and $d \geq 4$. The dense packing property of the Barnes-Wall lattices enables effective error correction. The encoding and decoding procedures are implemented by the functions **MsgEnc** and **MsgDec**, described in [Algorithm 2](#) and [Algorithm 4](#), respectively. Furthermore, **MsgEnc** calls the labeling function **Label** outlined in [Algorithm 3](#), while **MsgDec** calls the delabeling algorithm **Delabel** outlined in [Algorithm 5](#) and the bounded distance decoding (BDD) algorithm outlined in [Algorithm 6](#). We note that all functions presented in these algorithms apply only to BW_d , the procedures for RBW_d are similar and are given in [Appendix A](#).

Algorithm 2 The **MsgEnc** Function

Input: Message $\mathbf{m} \in \{0, 1\}^{l_m}$, power-of-two integer $q = 2^t$, matrix dimensions $\bar{m}, \bar{n}$
Input: Lattice BW_d , power-of-two integer $g = 2^\tau$, repetition number σ
Output: Matrix $\mathbf{M} \in \mathbb{Z}_q^{\bar{m} \times \bar{n}}$

- 1: Divide $\mathbf{m}$ into σ subvectors $\mathbf{m} = \mathbf{m}_1 || \mathbf{m}_2 || \dots || \mathbf{m}_\sigma$, where $\mathbf{m}_j \in \{0, 1\}^{l_m/\sigma}$
- 2: **for** $j = 1$ to σ **do**
- 3: Invoke **Label** with input BW_d , $g = 2^\tau$ and $\mathbf{m}_j$ to obtain a lattice vector $\mathbf{w}_j \in \mathbb{Z}[\mathbf{i}]^{d/2}$
- 4: Write $\mathbf{w}_j$ as $\mathbf{w}_j = \mathbf{u}_j + \mathbf{v}_j \mathbf{i}$, where $\mathbf{u}_j, \mathbf{v}_j \in \mathbb{Z}^{d/2}$
- 5: **end for**
- 6: $\mathbf{x} \leftarrow (\mathbf{u}_1, \mathbf{v}_1, \mathbf{u}_2, \mathbf{v}_2, \dots) \in \mathbb{Z}^{d \cdot \sigma}$
- 7: Append zeros to $\mathbf{x}$ to extend it to a vector $\mathbf{x}' \in \mathbb{Z}^{\bar{m}\bar{n}}$
- 8: $\mathbf{y} = (y_1, \dots, y_{\bar{m}\bar{n}}) \leftarrow (q/g) \cdot \mathbf{x}'$
- 9: Assemble matrix $\mathbf{M}$ by setting the (i, j) -th entry to $y_{\bar{n}(i-1)+j}$ for $1 \leq i \leq \bar{m}$ and $1 \leq j \leq \bar{n}$
- 10: **return** $\mathbf{M}$

Message Layout. For each BW/RBW code block, if $\mathbf{w}_j = \mathbf{u}_j + \mathbf{v}_j \mathbf{i}$, the implementation stores the coordinates in component-pair order as $(u_{j,1}, v_{j,1}, u_{j,2}, v_{j,2}, \dots)$, i.e., the real and imaginary parts of each component are placed adjacently. Repeated blocks are concatenated as $(\mathbf{w}_1, \mathbf{w}_2, \dots)$, padded with zeros to length $\bar{m}\bar{n}$, scaled by q/g , and placed into $\mathbf{M}$ in row-major order. The decoder uses the exact inverse row-major mapping before invoking BDD.

Algorithm 3 The **Label** Function

Input: Lattice BW_d with $d = 2^k$, $g = 2^\tau$
Input: $\mathbf{m} \in \{0, 1\}^*$ of length at most $\tau d - \frac{d}{4}(k-1)$
Output: A lattice vector $\mathbf{x} \in BW_d$

- 1: Pad $\mathbf{m}$ with zeros to obtain $\mathbf{m}' = \mathbf{m} || \mathbf{0} \in \{0, 1\}^{\tau d - \frac{d}{4}(k-1)}$
- 2: Divide $\mathbf{m}' = \mathbf{u}_0 || \mathbf{u}_1 || \dots || \mathbf{u}_{\frac{d}{2}-1}$ such that $\mathbf{u}_j \in \{0, 1\}^{2^{\tau-w_H(j)}}$
- 3: **for** j from 0 to $\frac{d}{2} - 1$ **do**
- 4: $v_j \leftarrow f_{2^{\tau-w_H(j)}}(\mathbf{u}_j) \in \mathbb{Z}[\mathbf{i}]$
- 5: **end for**
- 6: $\mathbf{v} \leftarrow (v_0, \dots, v_{\frac{d}{2}-1})$
- 7: **for** l from 1 to $k-1$ **do**
- 8: Write $\mathbf{v} = (\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_{\frac{d}{2^l}})$, where $\mathbf{w}_j \in \mathbb{Z}[\mathbf{i}]^{2^{l-1}}$
- 9: Update $\mathbf{v} \leftarrow (\mathbf{w}_1, \mathbf{w}_1 + \phi \mathbf{w}_2, \mathbf{w}_3, \mathbf{w}_3 + \phi \mathbf{w}_4, \dots, \mathbf{w}_{\frac{d}{2^{l-1}}}, \mathbf{w}_{\frac{d}{2^{l-1}}} + \phi \mathbf{w}_{\frac{d}{2^l}})$, where $\phi = 1 + \mathbf{i}$
- 10: **end for**
- 11: $\mathbf{w} \leftarrow \lceil \mathbf{v} \rceil_{2^\tau}$
- 12: **return** $\mathbf{w}$

The formal theory and proofs of **MsgEnc** and **MsgDec** using lattice coding are discussed in [Section 3](#). Here we provide a brief explanation.

In **MsgEnc**, the input message $\mathbf{m}$ is split equally into σ parts, and each part is mapped to a lattice vector $\mathbf{w}_j$ in BW_d (or RBW_d) using the labeling function **Label**. Note that in this document the Barnes-Wall lattices are represented as $\mathbb{Z}[\mathbf{i}]$ -submodules of $\mathbb{Z}[\mathbf{i}]^{d/2}$, thus the vectors $\{\mathbf{w}_j\}_{1 \leq j \leq \sigma}$ are converted into an integer vector by separating real and imaginary parts, and the resulting vector is then reshaped into a matrix.

The labeling function **Label** uses a map $f_l : \{0, 1\}^l \rightarrow \mathbb{Z}[\mathbf{i}]$ defined as follows. Given an input vector $\mathbf{u} \in \{0, 1\}^l$, f_l maps it to $f_l(\mathbf{u}) = a + b\mathbf{i}$ where $0 \leq a < 2^{\lceil l/2 \rceil}$ and $0 \leq b < 2^{\lfloor l/2 \rfloor}$. The first $\lceil l/2 \rceil$ bits of $\mathbf{u}$ encode a , and the remaining $\lfloor l/2 \rfloor$ bits encode b . Concretely, $a = \sum_{j=0}^{\lceil l/2 \rceil - 1} u_j \cdot 2^j$ and $b = \sum_{j=\lceil l/2 \rceil}^{l-1} u_j \cdot 2^j$, where $\mathbf{u} = (u_0, \dots, u_{l-1})$. Besides, it can be verified that the sum of the lengths of the vectors $\mathbf{u}_j$ in line 2 is $\sum_{0 \leq j < d/2} (2\tau - w_H(j)) = \tau d - \frac{d}{4}(k-1)$.

We note that assembling the matrix $\mathbf{M}$ requires $d \cdot \sigma \leq \bar{m} \cdot \bar{n}$, and invoking the **Label** function implicitly requires $\frac{l_m}{\sigma} \leq \tau d - \frac{d}{4}(k-1)$ for BW_d (or $\frac{l_m}{\sigma} \leq \tau d - \frac{d}{4}(k+1)$ for RBW_d), these conditions are satisfied by the parameter choices of Scloud⁺ in [Section 1.6](#).

Algorithm 4 The **MsgDec** Function

Input: Matrix $\mathbf{M} \in \mathbb{Z}_q^{\bar{m} \times \bar{n}}$, message length l_m

Input: Lattice BW_d , repetition number σ , and power-of-two integers $q = 2^t$, $g = 2^\tau$

Output: Message $\mathbf{m} \in \{0, 1\}^{l_m}$

- 1: $\mathbf{y} \leftarrow (y_1, y_2, \dots, y_{\bar{m}\bar{n}})$ such that $y_{\bar{n}(i-1)+j}$ is the (i, j) -th element of $\mathbf{M}$
 - 2: Truncate $\mathbf{y}$ to its first $d \cdot \sigma$ entries to form vector $\mathbf{x} \in \mathbb{Z}^{d \cdot \sigma}$
 - 3: $\mathbf{x} \leftarrow \frac{2^\tau}{q} \cdot \mathbf{x}$
 - 4: Partition $\mathbf{x}$ as $\mathbf{x} = (\mathbf{u}_1, \mathbf{v}_1, \mathbf{u}_2, \mathbf{v}_2, \dots)$ such that $\mathbf{u}_j, \mathbf{v}_j \in \mathbb{Z}^{d/2}$
 - 5: **for** $j = 1$ to σ **do**
 - 6: Invoke BDD on input target $\mathbf{u}_j + \mathbf{v}_j \mathbf{i}$ to obtain lattice vector $\mathbf{w}_j$ in BW_d
 - 7: Invoke **Delabel** with input BW_d , $g = 2^\tau$ and $[\mathbf{w}_j]_{2^\tau}$ to obtain $\mathbf{m}_j \in \{0, 1\}^{l_m/\sigma}$
 - 8: **end for**
 - 9: **return** $\mathbf{m} = \mathbf{m}_1 || \dots || \mathbf{m}_\sigma$
-

Algorithm 5 The **Delabel** Function

Input: Lattice BW_d with $d = 2^k$, $g = 2^\tau$

Input: A lattice vector $\mathbf{w} \in \text{BW}_d$

Output: $\mathbf{m} \in \{0, 1\}^*$

- 1: **for** l from $k-1$ to 1 **do**
 - 2: Write $\mathbf{w} = (\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_{d/2^l})$, where $\mathbf{w}_j \in \mathbb{Z}[\mathbf{i}]^{2^{l-1}}$ for $1 \leq j \leq d/2^l$
 - 3: Update $\mathbf{w} \leftarrow (\mathbf{w}_1, \phi^{-1}(\mathbf{w}_2 - \mathbf{w}_1), \mathbf{w}_3, \phi^{-1}(\mathbf{w}_4 - \mathbf{w}_3), \dots)$
 - 4: **end for**
 - 5: Write $\mathbf{w} = (a_0, \dots, a_{\frac{d}{2}-1}) + (b_0, \dots, b_{\frac{d}{2}-1}) \cdot \mathbf{i}$
 - 6: **for** j from 0 to $\frac{d}{2} - 1$ **do**
 - 7: $b'_j \leftarrow [b_j]_{2^\tau - \lceil w_H(j)/2 \rceil}$, $a'_j \leftarrow [a_j - (b_j - b'_j)]_{2^\tau - \lfloor w_H(j)/2 \rfloor}$, $\mathbf{u}_j \leftarrow f_{2^\tau - w_H(j)}^{-1}(a'_j + b'_j \mathbf{i})$
 - 8: **end for**
 - 9: **return** $\mathbf{m} = \mathbf{u}_0 || \mathbf{u}_1 || \dots || \mathbf{u}_{\frac{d}{2}-1}$
-

Algorithm 6 BDD Algorithm for Barnes-Wall lattices**Input:** A target vector $\mathbf{t} \in \mathbb{C}^{d/2}$ and lattice BW_d **Output:** A lattice vector $\mathbf{y} \in \text{BW}_d$

```

1: if  $d = 2$  then
2:   return  $\lfloor \mathbf{t} \rfloor$ 
3: else
4:   Write  $\mathbf{t} = (\mathbf{t}_1, \mathbf{t}_2)$  such that  $\mathbf{t}_1, \mathbf{t}_2 \in \mathbb{C}^{d/4}$ 
5:    $\mathbf{y}_1 \leftarrow \text{BDD}(\mathbf{t}_1, \text{BW}_{d/2})$ ,  $\mathbf{y}_2 \leftarrow \text{BDD}(\mathbf{t}_2, \text{BW}_{d/2})$ 
6:    $\mathbf{z}_1 \leftarrow \text{BDD}(\phi^{-1}(\mathbf{t}_2 - \mathbf{y}_1), \text{BW}_{d/2})$ ,  $\mathbf{z}_2 \leftarrow \text{BDD}(\phi^{-1}(\mathbf{t}_1 - \mathbf{y}_2), \text{BW}_{d/2})$ 
7:   Compute  $\mathbf{x} = (\mathbf{y}_1, \mathbf{y}_1 + \phi \mathbf{z}_1)$ ,  $\mathbf{x}' = (\mathbf{y}_2 + \phi \mathbf{z}_2, \mathbf{y}_2)$ 
8:   if  $\|\mathbf{x} - \mathbf{t}\| < \|\mathbf{x}' - \mathbf{t}\|$  then
9:     return  $\mathbf{x}$ 
10:  else
11:    return  $\mathbf{x}'$ 
12:  end if
13: end if

```

The **MsgDec** function reverses the encoding process of **MsgEnc** and additionally incorporates a bounded distance decoding (BDD) algorithm to perform error reconciliation. The BDD algorithm for Barnes-Wall lattices, given in [Algorithm 6](#), is a variant of the decoder proposed in [20].

We note that **MsgDec** and BDD may appear to involve floating-point computations. However, since all these computations can be viewed as operations over the fractional ring $\frac{1}{2^t}\mathbb{Z}$, **MsgDec** and BDD can be easily transformed into exact algorithms operating on integers. Versions of [Algorithm 4](#) and [Algorithm 6](#) that exclude floating-point operations are provided in [Appendix B](#).

1.4 Specification of Scloud⁺.PKE

Scloud⁺.PKE consists of three algorithms: key generation, encryption, and decryption, which are outlined in [Algorithm 7](#) to [Algorithm 9](#). The algorithms utilize the following parameters:

- Modulus: powers of 2 positive integer $q = 2^t$;
- Matrix size parameters: positive integers $m, n, \bar{m}, \bar{n}$;
- Secret and error parameters: ρ_s, ρ_e ;
- Message length: $l_m \in \{128, 192, 256, 384, 512\}$;
- Coding parameters: lattice BW_d or RBW_d , $g = 2^\tau$, repetition number σ .

Algorithm 7 Scloud⁺.PKE.KeyGen()**Output:** Public key $\mathbf{pk} \in \{0, 1\}^{m \cdot \bar{n} \cdot t + 128}$ **Output:** Secret key $\mathbf{sk} \in \{0, 1\}^{n \cdot \bar{n} \cdot 2}$

```

1:  $\alpha \leftarrow \{0, 1\}^h$ 
2: Generate  $(\text{seed}_A, \mathbf{r}_1, \mathbf{r}_2) \leftarrow \text{PRF}(\alpha)$  such that  $\text{seed}_A \in \{0, 1\}^{128}$ ,  $\mathbf{r}_1, \mathbf{r}_2 \in \{0, 1\}^h$ 
3:  $\mathbf{A} \leftarrow \text{gen}(\text{seed}_A) \in \mathbb{Z}_q^{m \times n}$ 
4:  $\mathbf{S} \leftarrow \text{BD}(\mathbf{r}_1, n, \bar{n}, \rho_s) \in \mathbb{Z}^{n \times \bar{n}}$ 
5:  $\mathbf{E} \leftarrow \text{BD}(\mathbf{r}_2, m, \bar{n}, \rho_e) \in \mathbb{Z}^{m \times \bar{n}}$ 
6:  $\mathbf{B} = \mathbf{A} \cdot \mathbf{S} + \mathbf{E} \bmod q$ 
7: return  $\mathbf{pk} = \text{Pack}(\mathbf{B}) \parallel \text{seed}_A$ ,  $\mathbf{sk} = \text{Packsk}(\mathbf{S})$ 

```

Algorithm 8 $\text{Scloud}^+.\text{PKE}.\text{Enc}(\mathbf{pk}, \mathbf{m}, \mathbf{r})$ **Input:** Public key $\mathbf{pk} \in \{0, 1\}^{m \cdot \bar{n} \cdot t + 128}$ **Input:** Message $\mathbf{m} \in \{0, 1\}^{l_m}$ **Input:** Random coins $\mathbf{r} \in \{0, 1\}^h$ **Output:** Ciphertext $\mathbf{ct} \in \{0, 1\}^{\bar{m} \cdot (n + \bar{n}) \cdot t}$

- 1: $(\mathbf{pk}', \text{seed}_A) \leftarrow \mathbf{pk}$
- 2: $\mathbf{A} \leftarrow \text{gen}(\text{seed}_A)$
- 3: $\mathbf{B} \leftarrow \text{Unpack}(\mathbf{pk}')$
- 4: Generate $(\mathbf{r}'_1, \mathbf{r}'_2) \leftarrow \text{PRF}(\mathbf{r})$ such that $\mathbf{r}'_1, \mathbf{r}'_2 \in \{0, 1\}^h$
- 5: $\mathbf{S}' \leftarrow \text{BD}(\mathbf{r}'_1, \bar{m}, m, \rho_s) \in \mathbb{Z}^{\bar{m} \times m}$
- 6: $(\mathbf{E}_1, \mathbf{E}_2) \leftarrow \text{BD}(\mathbf{r}'_2, \bar{m}, n + \bar{n}, \rho_e)$ such that $\mathbf{E}_1 \in \mathbb{Z}^{\bar{m} \times n}$, $\mathbf{E}_2 \in \mathbb{Z}^{\bar{m} \times \bar{n}}$
- 7: Invoke MsgEnc with input $\mathbf{m}$ and the coding parameters to obtain a matrix $\mathbf{M} \in \mathbb{Z}_q^{\bar{m} \times \bar{n}}$
- 8: $\mathbf{C}_1 = \mathbf{S}' \cdot \mathbf{A} + \mathbf{E}_1 \bmod q$
- 9: $\mathbf{C}_2 = \mathbf{S}' \cdot \mathbf{B} + \mathbf{E}_2 + \mathbf{M} \bmod q$
- 10: **return** $\mathbf{ct} = \text{Pack}(\mathbf{C}_1) \parallel \text{Pack}(\mathbf{C}_2)$

Algorithm 9 $\text{Scloud}^+.\text{PKE}.\text{Dec}(\mathbf{sk}, \mathbf{ct})$ **Input:** Secret key $\mathbf{sk} \in \{0, 1\}^{n \cdot \bar{n} \cdot 2}$ **Input:** Ciphertext $\mathbf{ct} \in \{0, 1\}^{\bar{m} \cdot (n + \bar{n}) \cdot t}$ **Output:** Message $\mathbf{m} \in \{0, 1\}^{l_m}$

- 1: $(\mathbf{C}'_1, \mathbf{C}'_2) \leftarrow \mathbf{ct}$
- 2: $\mathbf{C}_1 \leftarrow \text{Unpack}(\mathbf{C}'_1)$, $\mathbf{C}_2 \leftarrow \text{Unpack}(\mathbf{C}'_2)$
- 3: $\mathbf{S} \leftarrow \text{Unpacksk}(\mathbf{sk})$
- 4: $\mathbf{D} = \mathbf{C}_2 - \mathbf{C}_1 \mathbf{S} \bmod q$
- 5: Invoke MsgDec with input $\mathbf{D}$ and the coding parameters to obtain a vector $\mathbf{m} \in \{0, 1\}^{l_m}$
- 6: **return** $\mathbf{m}$

1.5 Specification of $\text{Scloud}^+.\text{KEM}$

$\text{Scloud}^+.\text{KEM}$ is derived from $\text{Scloud}^+.\text{PKE}$ by applying the Fujisaki-Okamoto transformation with implicit rejection [3]. $\text{Scloud}^+.\text{KEM}$ consists of three algorithms: key generation, encapsulation, and decapsulation, which are described in [Algorithm 10](#) to [Algorithm 12](#).

Algorithm 10 $\text{Scloud}^+.\text{KEM}.\text{KeyGen}()$ **Output:** Public key $\mathbf{pk} \in \{0, 1\}^{m \cdot \bar{n} \cdot t + 128}$ **Output:** Secret key $\mathbf{sk} \in \{0, 1\}^{m \cdot \bar{n} \cdot t + n \cdot \bar{n} \cdot 2 + 128 + h \times 2}$

- 1: $(\mathbf{pk}, \mathbf{sk}') \leftarrow \text{Scloud}^+.\text{PKE}.\text{KeyGen}()$
- 2: $\mathbf{hpk} = \mathbf{H}(\mathbf{pk}) \in \{0, 1\}^h$, $\mathbf{z} \leftarrow U(\{0, 1\}^h)$
- 3: $\mathbf{sk} = \mathbf{sk}' \parallel \mathbf{pk} \parallel \mathbf{hpk} \parallel \mathbf{z}$
- 4: **return** $(\mathbf{pk}, \mathbf{sk})$

Algorithm 11 $\text{Scloud}^+.\text{KEM}.\text{Encaps}()$ **Input:** Public key $\mathbf{pk} \in \{0, 1\}^{m \cdot \bar{n} \cdot t + 128}$ **Output:** Ciphertext $\mathbf{ct} \in \{0, 1\}^{\bar{m} \cdot (n + \bar{n}) \cdot t}$ **Output:** Shared session key $\mathbf{ss} \in \{0, 1\}^{l_{ss}}$

- 1: $\mathbf{m} \leftarrow U(\{0, 1\}^{l_{ss}})$
- 2: $(\mathbf{r}, \mathbf{k}) = \mathbf{G}(\mathbf{m} \parallel \mathbf{H}(\mathbf{pk})) \in \{0, 1\}^{h \times 2}$
- 3: $\mathbf{ct} \leftarrow \text{Scloud}^+.\text{PKE}.\text{Enc}(\mathbf{pk}, \mathbf{m}, \mathbf{r})$
- 4: $\mathbf{ss} = \text{KDF}(\mathbf{k} \parallel \mathbf{ct}) \in \{0, 1\}^{l_{ss}}$
- 5: **return** $(\mathbf{ct}, \mathbf{ss})$

Algorithm 12 $\text{Scloud}^+.\text{KEM}.\text{Decaps}()$

Input: Ciphertext $\text{ct} \in \{0, 1\}^{\bar{m} \cdot (n + \bar{n}) \cdot t}$
Input: Secret key $\text{sk} \in \{0, 1\}^{m \cdot \bar{n} \cdot t + n \cdot \bar{n} \cdot 2 + 128 + h \times 2}$
Output: Shared session key $\text{ss} \in \{0, 1\}^{l_{\text{ss}}}$

- 1: $(\text{sk}', \text{pk}, \text{hpk}, \text{z}) \leftarrow \text{sk}$
- 2: $\text{m}' = \text{Scloud}^+.\text{PKE}.\text{Dec}(\text{sk}', \text{ct})$
- 3: $(\text{r}', \text{k}') = \mathcal{G}(\text{m}' || \text{hpk}) \in \{0, 1\}^{h \times 2}$
- 4: $\text{ct}' = \text{Scloud}^+.\text{PKE}.\text{Enc}(\text{pk}, \text{m}', \text{r}')$
- 5: **if** $\text{ct} = \text{ct}'$ **then**
- 6: **return** $\text{ss} = \text{KDF}(\text{k}' || \text{ct})$
- 7: **else**
- 8: **return** $\text{ss} = \text{KDF}(\text{z} || \text{ct})$
- 9: **end if**

1.6 Parameter Sets of Scloud^+

We provide five parameter sets for Scloud^+ , targeting classical security levels of $\lambda = 128, 192, 256, 384$, and 512 bits, respectively. The parameter sets are listed in [Table 2](#), where the modulus q is fixed to 2^{10} for all sets. The length of the message/shared secret is set equal to the target security level. All m and n are set to multiples of 32 for implementation convenience. The level-dependent hash-material length h is defined as

$$h = \begin{cases} 512, & \text{for } \lambda \in \{128, 192, 256\}, \\ 768, & \text{for } \lambda = 384, \\ 1024, & \text{for } \lambda = 512. \end{cases}$$

Table 2. Parameters for $\text{Scloud}^+.\text{PKE}$ and $\text{Scloud}^+.\text{KEM}$.

	$l_{\text{ss}} = l_m$	q	(m, n)	$(\bar{m}, \bar{n})$	ρ_s	ρ_e
Scloud^+-128	128	2^{10}	(608, 608)	(8, 8)	1/4	1/2
Scloud^+-192	192	2^{10}	(832, 832)	(12, 11)	1/2	1/2
Scloud^+-256	256	2^{10}	(1184, 1184)	(12, 11)	1/6	1/6
Scloud^+-384	384	2^{10}	(1664, 1664)	(16, 16)	1/6	1/6
Scloud^+-512	512	2^{10}	(2400, 2400)	(16, 16)	1/12	1/12

The coding parameters are provided in [Table 3](#). For Scloud^+-128 , the coding lattice is BW_{32} ; for Scloud^+-192 and Scloud^+-384 , it is BW_{128} ; and for Scloud^+-256 and Scloud^+-512 , it is RBW_{128} . We note that for all these parameter sets, the failure probability is below $2^{-\lambda}$ (with some margin), where λ denotes the target security level. This is discussed in more detail in [Section 3](#).

Table 3. Coding parameters for MsgEnc and MsgDec .

	Coding lattice	$g = 2^r$	σ
Scloud^+-128	BW_{32}	2^3	2
Scloud^+-192	BW_{128}	2^3	1
Scloud^+-256	RBW_{128}	2^4	1
Scloud^+-384	BW_{128}	2^3	2
Scloud^+-512	RBW_{128}	2^4	2

1.6.1 Key and Ciphertext Sizes. Table 4 presents the sizes (in bytes) of the public key, ciphertext, and shared secret for Scloud⁺.KEM.

Table 4. Public key, ciphertext, and shared secret sizes for Scloud⁺.KEM (in bytes).

Scheme	Public key pk	Ciphertext C	Shared secret ss
Scloud ⁺ .KEM-128	6096	6160	16
Scloud ⁺ .KEM-192	11456	12645	24
Scloud ⁺ .KEM-256	16296	17925	32
Scloud ⁺ .KEM-384	33296	33600	48
Scloud ⁺ .KEM-512	48016	48320	64

2 Performance Analysis

In this section, we provide the remaining implementation details for Scloud⁺ and present the performance results of Scloud⁺.KEM.

2.1 Instantiations for the Symmetric Primitives

We provide three algorithm families for the instantiation of the symmetric primitives PRF, XOF, H, G, and KDF, referred to as the AES, SHAKE, and SM3 families, respectively. Across all families, public matrix generation uses a 128-bit seed **seed_A**. The FO transform uses the level-dependent hash-material length h : $h = 512$ for Scloud⁺-128, Scloud⁺-192, and Scloud⁺-256, $h = 768$ for Scloud⁺-384, and $h = 1024$ for Scloud⁺-512. The output of G is parsed as $r \parallel k$ with $|r| = |k| = h$.

For the AES and SHAKE families, the functions PRF (or F), G, and KDF (or K) are instantiated by SHAKE-256. The function H is SHA3-512 when $h = 512$; for $h > 512$, where no fixed-output SHA3 variant has the required length, H is instantiated by SHAKE-256 with h output bits. For the SM3 family, PRF, G, and KDF use the submitted SM3 pseudo-XOF, and H uses the submitted SM3 pseudo-hash interface at the required output length.

The implementation domain-separates the four FO primitives by prepending one fixed public byte to the input before invoking the underlying hash/XOF:

$$F : 0x46, \quad G : 0x47, \quad H : 0x48, \quad K : 0x4b.$$

These are the ASCII bytes for “F”, “G”, “H”, and “K”. This byte is not a nonce and is not sampled. It is a fixed label that separates the different random-oracle roles when the same concrete primitive family is used underneath. For AES/SHAKE builds the label is absorbed as the first byte of SHAKE-256 or SHA3-512. For SM3 builds the label is prepended to the message passed to the SM3 pseudo-XOF or pseudo-hash. This also makes the implementation convention explicit for the FGHK-style transform: the proof treats F, G, H, K as separate oracles, while the code realizes this separation with these four labels.

Public matrix generation is family-specific: the AES family uses AES-128 in CTR mode, the SHAKE family uses SHAKE-128, and the SM3 family uses the SM3 pseudo-XOF. The concrete instantiations are summarized in Table 5.

All entries in Table 6 are bytes. The “ h -bit strings” column covers the PKE key-generation coins α , the sampling seeds r_1, r_2, r'_1, r'_2 , the PKE encryption coins r , the re-encryption key material k , $H(pk)$, and the implicit-rejection secret z . The random FO message m and the final shared secret ss use `scloudplus_ss` bytes and therefore have length λ bits, not h bits. The output of $F(\alpha)$ is **seed_A** $\parallel r_1 \parallel r_2$, while the output of $F(r)$ is $r'_1 \parallel r'_2$.

Table 5. Instantiation of symmetric primitives per algorithm family.

Primitive AES family		SHAKE family	SM3 family
XOF for A	AES-128 CTR	SHAKE-128	SM3 pseudo-XOF
PRF / F	labeled SHAKE-256	labeled SHAKE-256	labeled SM3 pseudo-XOF
H	labeled SHA3-512 if $h = 512$; labeled SHAKE-256 if $h > 512$	same as AES family	labeled SM3 pseudo-hash
G	labeled SHAKE-256, $2h$ bits	labeled SHAKE-256, $2h$ bits	labeled SM3 pseudo-XOF, $2h$ bits
KDF / K	labeled SHAKE-256	labeled SHAKE-256	labeled SM3 pseudo-XOF

Table 6. Implementation byte lengths for random and derived material.

Parameter set	m, ss	h -bit strings	seed _A	G output	$F(\alpha)$ output	$F(r)$ output
Scld ⁺ -128	16	64	16	128	144	128
Scld ⁺ -192	24	64	16	128	144	128
Scld ⁺ -256	32	64	16	128	144	128
Scld ⁺ -384	48	96	16	192	208	192
Scld ⁺ -512	64	128	16	256	272	256

2.2 Matrix Generation and Sampling

Public matrix generation. The public matrix $\mathbf{A} \in \mathbb{Z}_q^{m \times n}$ is generated in a family-specific manner and represented in *packed10* format. Since $q = 2^{10}$, each row needs $L_A = 10n/8$ bytes before unpacking. Let $N_A = \lceil L_A/16 \rceil$. For row i , the byte stream is generated as follows:

- AES family: AES-128 in CTR mode under key seed_A . The implementation encrypts the zero-padded counter blocks $\text{LE32}(iN_A + j) \parallel 0^{96}$ for $0 \leq j < N_A$ and ignores any padding bytes after the first L_A bytes;
- SHAKE family: $\text{SHAKE128}(\text{seed}_A \parallel \text{LE32}(i), L_A)$ for row i ;
- SM3 family: $\text{SM3-pseudoXOF}(\text{seed}_A \parallel \text{LE32}(i), L_A)$ for row i , with the pseudo-XOF internal output counter starting from 1.

The resulting byte string is parsed into coefficients of $\mathbb{Z}_{2^{10}}$ by little-endian 10-bit parsing and arranged in row-major order. Each consecutive five bytes are decoded little-endian into four coefficients:

$$\begin{aligned}
 a_0 &= b_0 + 2^8(b_1 \bmod 2^2), \\
 a_1 &= \lfloor b_1/2^2 \rfloor + 2^6(b_2 \bmod 2^4), \\
 a_2 &= \lfloor b_2/2^4 \rfloor + 2^4(b_3 \bmod 2^6), \\
 a_3 &= \lfloor b_3/2^6 \rfloor + 2^2b_4.
 \end{aligned}$$

Implementation parsing of BD samples. The implementation fills matrices by consuming a deterministic byte stream from the selected sampling XOF. For AES and SHAKE families, SHAKE-128 is used for Scld⁺-128, Scld⁺-192 and SHAKE-256 is used for Scld⁺-256, Scld⁺-384, Scld⁺-512; for the SM3 family the bounded SM3 pseudo-XOF is used for all four values, with its internal output counter starting from 1. The coefficients are stored as 16-bit integers with -1 represented by $2^{16} - 1$. This sampling XOF is separate from the labeled FO primitive F : F expands the PKE or KEM coins into sampling seeds, while XOF_{BD} expands those seeds into the byte stream parsed by BD2, BD4, BD6, or BD12.

For $p = 2$, each coefficient consumes two bits. If the two-bit value is b_1b_0 , then the coefficient is $b_1 - b_0$; hence one byte gives four coefficients from bit pairs $(1, 0), (3, 2), (5, 4), (7, 6)$. For $p = 4$, each coefficient consumes one nibble. The low two bits define the left Bernoulli trial by testing whether they are zero, and the high two bits define the right trial in the same way; the coefficient is their difference. For $p = 6$, the stream is read as little-endian 3-bit candidates. Values $0, \dots, 5$ are accepted, values $6, 7$ are rejected, and an accepted candidate contributes the Bernoulli bit ($a = 0$). Two accepted bits form one coefficient. For $p = 12$, the same method is applied to 4-bit candidates: values $0, \dots, 11$ are accepted, values $12, \dots, 15$ are rejected, and low nibbles are processed before high nibbles.

The secret matrices $\mathbf{S}$ and $\mathbf{S}'$ use ρ_s , while $\mathbf{E}, \mathbf{E}_1, \mathbf{E}_2$ use ρ_e . The implementation stores every sampled matrix as one linear array. For the long-term secret $\mathbf{S}$, the entry at offset $kn + j$ is

interpreted as $S_{j,k}$ in the $\mathbf{AS}$ and $C_1\mathbf{S}$ products, i.e., the linear array is column-major with respect to the mathematical $n \times \bar{n}$ matrix. The same linear array is serialized by **ShortPack** four coefficients per byte. The encryption secret $\mathbf{S}'$ and the error matrices are interpreted in the row-major order used by their products. For encryption, $\mathbf{E}_1$ and $\mathbf{E}_2$ are drawn from one continuous BD stream: all entries of $\mathbf{E}_1$ are generated first, followed by all entries of $\mathbf{E}_2$.

2.3 Implementation Results

All reported values are averaged over 5 independent runs. The x86_64 timings are measured in 10^3 CPU cycles on Linux 6.8.9 with an Intel Xeon Gold 6248R at 3.00 GHz, GCC 13.2.1, and CMake 3.27.7 in Release mode. The Linux AArch64 timings are measured in microseconds on a HiSilicon Kunpeng 920 machine running Ubuntu 22.04 with Linux 5.15.0-94-generic, GCC 11.4.0. The macOS arm64 timings are measured in microseconds on an Apple M3 Max with Apple Clang 16.0.0 and CMake 4.3.2. This section presents results for the optimized implementation only, and reference implementation results are referred to the appendix.

2.3.1 AES-based Results. Table 7, Table 8, and Table 9 present the optimized performance of Scloud⁺.KEM (AES family) on x86_64, Linux AArch64, and macOS arm64, respectively.

Table 7. The x86_64 optimized performance of Scloud⁺.KEM with AES, measured in 10^3 cycles.

Level	KeyGen	Encaps	Decaps
128	791.1	838.5	752.2
192	1561.2	1715.8	1658.7
256	3510.9	3828.6	3719.8
384	7490.8	7876.6	7638.5
512	14426.6	14792.6	14552.5

Table 8. The Linux AArch64 optimized performance of Scloud⁺.KEM with AES, measured in microseconds.

Level	KeyGen	Encaps	Decaps
128	241.8	246.2	218.5
192	471.8	523.2	492.2
256	1012.9	1089.0	1035.1
384	2388.3	2401.8	2355.6
512	4558.5	4639.2	4554.6

Table 9. The macOS arm64 optimized performance of Scloud⁺.KEM with AES, measured in microseconds.

Level	KeyGen	Encaps	Decaps
128	127.8	114.3	92.3
192	253.3	244.1	243.3
256	495.9	474.8	456.6
384	1298.9	1010.4	1013.7
512	2745.2	1978.8	1997.3

2.3.2 SHAKE-based Results. Table 10, Table 11, and Table 12 present the optimized performance of Scloud⁺.KEM (SHAKE family) on x86_64, Linux AArch64, and macOS arm64, respectively.

Table 10. The x86_64 optimized performance of Scloud⁺.KEM with SHAKE, measured in 10³ cycles.

Level	KeyGen	Encaps	Decaps
128	1805.6	1854.1	1772.2
192	3541.3	3662.7	3593.0
256	7006.7	7319.2	7229.8
384	14742.6	15035.3	14742.6
512	28141.9	28740.9	28468.5

Table 11. The Linux AArch64 optimized performance of Scloud⁺.KEM with SHAKE, measured in microseconds.

Level	KeyGen	Encaps	Decaps
128	1258.4	1278.4	1247.6
192	2405.5	2464.1	2430.3
256	4586.5	4666.6	4604.4
384	9410.7	9432.6	9371.8
512	18757.7	18844.0	18727.8

Table 12. The macOS arm64 optimized performance of Scloud⁺.KEM with SHAKE, measured in microseconds.

Level	KeyGen	Encaps	Decaps
128	625.9	611.1	586.0
192	1161.9	1157.1	1152.3
256	2204.7	2181.3	2171.1
384	4612.8	4345.3	4307.0
512	9218.8	8669.5	8523.1

2.3.3 SM3-based Results. [Table 13](#), [Table 14](#), and [Table 15](#) present the optimized performance of Scloud⁺.KEM (SM3 family) on x86_64, Linux AArch64, and macOS arm64, respectively.

Table 13. The x86_64 optimized performance of Scloud⁺.KEM with SM3, measured in 10³ cycles.

Level	KeyGen	Encaps	Decaps
128	3707.0	3861.7	3638.0
192	6907.7	7197.1	6844.4
256	14222.3	14591.7	14092.8
384	28475.9	28840.0	27872.9
512	55960.7	56402.6	54037.2

Table 14. The Linux AArch64 optimized performance of Scloud⁺.KEM with SM3, measured in microseconds.

Level	KeyGen	Encaps	Decaps
128	2241.5	2246.9	2229.8
192	4247.9	4276.3	4266.6
256	8673.4	8735.2	8718.4
384	17295.4	17280.7	17190.6
512	35721.6	35721.8	35574.1

Table 15. The macOS arm64 optimized performance of Scloud⁺.KEM with SM3, measured in microseconds.

Level	KeyGen	Encaps	Decaps
128	3169.8	3161.8	3131.3
192	5937.7	5954.0	5958.6
256	12128.8	11674.0	11544.6
384	24984.0	23397.3	23263.3
512	48956.6	49220.2	48780.5

3 Lattice Coding and DFR Analysis

In this section, we first introduce fundamental concepts related to lattice codes and Barnes-Wall lattices. We then present proofs for the lattice coding used in Scloud⁺. Finally, we provide a detailed analysis of the decryption failure rate of Scloud⁺.

3.1 Lattice Codes and Barnes-Wall Lattices

To begin with, we specify some fundamental definitions and notations related to lattices. A lattice $\mathcal{L}$ of rank m and dimension n (with $m \leq n$) is a discrete subset of $\mathbb{R}^n$ defined as

$$\mathcal{L} = \{c_1 \mathbf{b}_1 + \cdots + c_m \mathbf{b}_m \mid c_i \in \mathbb{Z} \text{ for } 1 \leq i \leq m\}, \quad (1)$$

where $\mathbf{b}_1, \dots, \mathbf{b}_m$ are linearly independent vectors in $\mathbb{R}^n$. The matrix $\mathbf{B} = (\mathbf{b}_1, \dots, \mathbf{b}_m)$ is called a *basis* for $\mathcal{L}$, and we denote the lattice generated by $\mathbf{B}$ as $\mathcal{L}(\mathbf{B})$. A lattice $\mathcal{L}$ is said to be of *full rank* if $m = n$. We use $\lambda_1(\mathcal{L})$ to denote the norm of shortest non-zero lattice vector in $\mathcal{L}$.

The *fundamental parallelepiped* of a basis $\mathbf{B}$ is defined as

$$\mathcal{P}(\mathbf{B}) = \{a_1 \mathbf{b}_1 + \cdots + a_m \mathbf{b}_m \mid a_i \in [0, 1) \text{ for } 1 \leq i \leq m\}.$$

Definition 2 (CVP). Given a lattice $\mathcal{L}$ and a target vector $\mathbf{t} \in \mathbb{R}^n$, the *Closest Vector Problem (CVP)* asks to find a lattice point $\mathbf{v} \in \mathcal{L}$ that is closest to $\mathbf{t}$, i.e., $\|\mathbf{v} - \mathbf{t}\| \leq \|\mathbf{v}' - \mathbf{t}\|$ for all $\mathbf{v}' \in \mathcal{L}$.

Definition 3 (BDD). Given a lattice $\mathcal{L}$ and a target vector $\mathbf{t} \in \mathbb{R}^n$ such that $\text{dist}(\mathbf{t}, \mathcal{L}) \leq r$ for problem parameter radius r , the *Bounded Distance Decoding (BDD)* problem asks to find a lattice point $\mathbf{v} \in \mathcal{L}$ that is closest to $\mathbf{t}$, i.e., $\|\mathbf{v} - \mathbf{t}\| \leq \|\mathbf{v}' - \mathbf{t}\|$ for all $\mathbf{v}' \in \mathcal{L}$.

Algorithms solve the above two problems are typically referred as Maximum Likelihood Decoding (MLD) algorithm and BDD algorithm (with decoding radius r) respectively.

3.1.1 Lattice Codes.

Definition 4 (Lattice Code). Let $\mathcal{L}_s, \mathcal{L}_c \subseteq \mathbb{R}^n$ be two full-rank lattices such that $\mathcal{L}_s \subseteq \mathcal{L}_c$. A *lattice code* $\mathcal{C}$ (based on the nested lattices $\mathcal{L}_s \subseteq \mathcal{L}_c$) is defined as a subset of $\mathcal{L}_c$ that forms a complete set of representatives for the quotient group $\mathcal{L}_c/\mathcal{L}_s$.

Given a basis $\mathbf{B}_s$ for $\mathcal{L}_s$, the set $\mathcal{C}$ is typically chosen to lie within the fundamental parallelepiped $\mathcal{P}(\mathbf{B}_s)$. The lattices $\mathcal{L}_s$ and $\mathcal{L}_c$ are commonly referred to as the *shaping lattice* and *coding lattice*, respectively.¹

In Scloud⁺, when a lattice code $\mathcal{C}$ is employed, the following steps are typically involved:

- *Labeling:* Each message $\mathbf{m} \in \mathcal{M}$ is mapped to a lattice vector $\mathbf{x} \in \mathcal{C}$, where $\mathcal{M}$ is the message space.
- *Lattice Decoding:* Given a noisy received vector $\mathbf{y} = \mathbf{x} + \mathbf{e} \in \mathbb{R}^n$, where $\mathbf{e}$ represents noise, this step employs an MLD or BDD algorithm. The algorithm takes $\mathbf{y}$ as the target vector and $\mathcal{L}_c$ as the lattice, producing a lattice vector $\mathbf{x}' \in \mathcal{C}$.

¹ In some literature, these are also called the *coarse lattice* and *fine lattice*.

- *Delabeling*: The decoded lattice vector $\mathbf{x}' \in \mathcal{L}_c$ is mapped back to a message $\mathbf{m}' \in \mathcal{M}$, essentially reversing the labeling step.

Note that in the lattice decoding step, the output $\mathbf{x}'$ of the MLD or BDD algorithm may not initially fall within the lattice code $\mathcal{C}$. In such cases, an additional operation is required to reduce $\mathbf{x}'$ modulo $\mathcal{L}_s$ to ensure it belongs to $\mathcal{C}$. If $\mathcal{C}$ is chosen to lie within the fundamental parallelepiped $\mathcal{P}(\mathbf{B}_s)$, this reduction can be achieved by expressing $\mathbf{x}'$ as linear combination of the basis $\mathbf{B}_s$ and reducing the coefficients to lie within $\mathcal{P}(\mathbf{B}_s)$.

A sufficient condition for correct decoding (i.e., $\mathbf{m} = \mathbf{m}'$) is that the labeling function is injective, and the noise vector $\mathbf{e}$ lies either within the Voronoi cell of $\mathcal{L}_c$ (if an MLD algorithm is employed) or within a decoding radius r (if a BDD algorithm with radius r is employed). To guarantee injectivity of the labeling map, we require that $|\mathcal{M}| \leq |\mathcal{C}| = \frac{\det(\mathcal{L}_s)}{\det(\mathcal{L}_c)}$. In this work, we focus on the message space $\mathcal{M}$ consisting of all bit strings of length μ , which necessitates $\mu = \log_2(|\mathcal{M}|) \leq \log_2\left(\frac{\det(\mathcal{L}_s)}{\det(\mathcal{L}_c)}\right)$. The quantity $\frac{1}{n} \log_2\left(\frac{\det(\mathcal{L}_s)}{\det(\mathcal{L}_c)}\right)$ is referred to as the *code rate* of $\mathcal{C}$, representing the average number of encoded bits per dimension.

3.1.2 Barnes-Wall Lattices. The Barnes-Wall lattices BW_d (or their rotations RBW_d) form a sequence of lattices defined for dimensions d that are powers of 2. In addition to their lattice structure, it is known that Barnes-Wall lattices can be viewed as $\mathbb{Z}[\mathbf{i}]$ -submodules of $\mathbb{Z}[\mathbf{i}]^{n/2}$, i.e., lattices over the Gaussian integers $\mathbb{Z}[\mathbf{i}]$ [21]. In this document, we adopt this construction to simplify the proofs of the labeling and delabeling processes. These lattices can be easily converted to lattices over the integers via a mapping from $\mathbb{C}^{d/2}$ to $\mathbb{R}^d$, as follows:

$$(a_1 + b_1\mathbf{i}, a_2 + b_2\mathbf{i}, \dots, a_{d/2} + b_{d/2}\mathbf{i}) \mapsto (a_1, b_1, a_2, b_2, \dots, a_{d/2}, b_{d/2}), \quad (2)$$

where $a_i, b_i \in \mathbb{R}$ for $1 \leq i \leq d/2$. Under this mapping, we naturally extend the norm and distance notations, defined in Section 1 for real vectors, to complex vectors.

Throughout the remainder of this section, we denote $\phi = 1 + \mathbf{i}$, so that $\phi^{-1} = \frac{1}{2}(1 - \mathbf{i}) = \frac{1}{2}\bar{\phi}$. The Barnes-Wall lattices can be defined recursively as follows.

Definition 5 (Barnes-Wall lattice). For any positive integer $d = 2^k \geq 4$, the d -dimensional Barnes-Wall lattice BW_d is defined as

$$\text{BW}_d = \{[\mathbf{u}, \mathbf{u} + \phi\mathbf{v}] \mid \mathbf{u}, \mathbf{v} \in \text{BW}_{d/2}\}, \quad (3)$$

with the initial case $\text{BW}_2 = \mathbb{Z}[\mathbf{i}]$. The rotated Barnes-Wall lattice RBW_d is defined as

$$\text{RBW}_d = \phi \cdot \text{BW}_d = \{\phi \cdot \mathbf{x} \mid \mathbf{x} \in \text{BW}_d\}. \quad (4)$$

Alternatively, it can be deduced that the Barnes-Wall lattices can also be expressed as

$$\text{BW}_n = \{\mathbf{W}_n \cdot \mathbf{v} \mid \mathbf{v} \in \mathbb{Z}[\mathbf{i}]^{\frac{n}{2}}\}, \text{ where } \mathbf{W}_n = \begin{pmatrix} 1 & 0 \\ 1 & \phi \end{pmatrix}^{\otimes(k-1)} \in \mathbb{C}^{\frac{n}{2} \times \frac{n}{2}}. \quad (5)$$

Here $\mathbf{A}^{\otimes m}$ denotes the self-Kronecker product of m copies of matrix $\mathbf{A}$.

For $n = 4, 8$, the Barnes-Wall lattices correspond, under the mapping defined in (2), to D_4 and E_8 respectively. These are known to be the densest packings in their respective dimensions.

Lemma 1 ([22]). For the (rotated) Barnes-Wall lattices with $d = 2^k \geq 2$, the following properties hold:

- (1) For BW_d , the minimum distance $\lambda_1 = \sqrt{\frac{d}{2}}$, and the packing radius $\rho = \frac{1}{2}\lambda_1 = \sqrt{\frac{d}{8}}$.
- (2) For RBW_d , the minimum distance $\lambda_1 = \sqrt{d}$, and the packing radius $\rho = \frac{1}{2}\lambda_1 = \sqrt{\frac{d}{4}}$.
- (2) The determinant $\det(\text{BW}_d) = 2^{\frac{d}{4}(k-1)}$, $\det(\text{RBW}_d) = 2^{\frac{d}{4}(k+1)}$.
- (3) $\phi^{k-1} \cdot \mathbb{Z}[\mathbf{i}]^{\frac{d}{2}} \subseteq \text{BW}_d$, and $\phi^k \cdot \mathbb{Z}[\mathbf{i}]^{\frac{d}{2}} \subseteq \text{RBW}_d$

3.1.3 Lattice Coding Based on Barnes-Wall Lattices. Their are two types of lattice codes involved in Scloud^+ , which are specified as follows.

Using BW_d as the Coding Lattice. This case covers the cases of Scloud^+ -128, Scloud^+ -192 and Scloud^+ -384, of which Barnes-Wall lattices are adopted as the coding lattice.

For $d = 2^k \geq 4$, let the coding lattice $\mathcal{L}_c = \text{BW}_d$ and the shaping lattice $\mathcal{L}_s = 2^\tau \cdot \mathbb{Z}[\mathbf{i}]^{d/2}$, where $\tau \geq \lfloor \frac{k}{2} \rfloor$ is a positive integer. Then by Lemma 1, it has $\mathcal{L}_s \subseteq 2^{\lfloor \frac{k}{2} \rfloor} \cdot \mathbb{Z}[\mathbf{i}]^{d/2} \subseteq \mathcal{L}_c$. Denote $\mathcal{C}$ to be the lattice code defined based on the nested lattices $\mathcal{L}_s \subseteq \mathcal{L}_c$, where each vector of $\mathcal{C}$ is required to be within the region $\mathcal{P} = \{(a_1 + b_1\mathbf{i}, a_2 + b_2\mathbf{i}, \dots, a_{d/2} + b_{d/2}\mathbf{i}) \mid 0 \leq a_j, b_j < 2^\tau \text{ for all } 1 \leq j \leq d/2\}$, i.e., the fundamental parallelepiped with respect to the standard basis of $\mathcal{L}_s = 2^\tau \cdot \mathbb{Z}[\mathbf{i}]^{d/2}$. By Lemma 1 it has

$$|\mathcal{C}| = \frac{\det(\mathcal{L}_s)}{\det(\mathcal{L}_c)} = \frac{2^{\tau d}}{2^{\frac{d}{4}(k-1)}} = 2^{\tau d - \frac{d}{4}(k-1)}. \quad (6)$$

Then we can prove the following lemma for the **Label** and **Delabel** functions defined in Algorithm 3 and Algorithm 5.

Lemma 2. Assume $\mu \leq \tau n - \frac{n}{4}(k-1)$, then the labeling function **Label** in Algorithm 3 defines an injective map from the message space $\mathcal{M} = \{0, 1\}^\mu$ to the lattice code $\mathcal{C}$. Conversely, the delabeling function **Delabel** in Algorithm 5 is the inverse of the labeling in Algorithm 3.

Proof. We first prove that **Label** is injective. Recall that the **Label** function can be divided into a two-step process. The first step maps $\mathbf{m}'$ to a vector $\mathbf{v} \in \mathbb{Z}[\mathbf{i}]^{d/2}$ using the map f_l , which corresponds to lines 2 to 6 in Algorithm 3. The second step maps $\mathbf{v}$ to a lattice vector $\mathbf{w} \in \mathcal{C}$, which corresponds to lines 7 to 11 in Algorithm 3.

Let $\mathcal{S}_l$ denote the image space of f_l , i.e., $\mathcal{S}_l = \{a + b\mathbf{i} \mid a, b \in \mathbb{Z}, 0 \leq a < 2^{\lceil l/2 \rceil}, 0 \leq b < 2^{\lfloor l/2 \rfloor}\}$. Clearly, the first step of **Label** defines an injective map from the message space $\mathcal{M}$ to the product space $\prod_{0 \leq j < d/2} \mathcal{S}_{2\tau - w_H(j)}$.

By the tensor product structure of $\mathbf{W}_n$, the second step iteratively computes $\mathbf{x}' = \mathbf{W}_n \cdot \mathbf{v}$, where $\mathbf{W}_n$ is defined in (5), and the rounding operation $\mathbf{x} = \lfloor \mathbf{x}' \rfloor_{2^\tau}$ yields a lattice vector in the lattice code $\mathcal{C}$. We now show that, for $\mathbf{v} \in \prod_{0 \leq j < d/2} \mathcal{S}_{2\tau - w_H(j)}$, the map $\mathbf{v} \mapsto \lfloor \mathbf{W}_d \cdot \mathbf{v} \rfloor_{2^\tau}$ is injective. Suppose there exist vectors $\mathbf{v}, \mathbf{v}' \in \prod_{0 \leq j < d/2} \mathcal{S}_{2\tau - w_H(j)}$ such that $\mathbf{W}_d \cdot \mathbf{v} = \mathbf{W}_d \cdot \mathbf{v}' \pmod{2^\tau}$, i.e., $\mathbf{W}_d \cdot (\mathbf{v} - \mathbf{v}') = 0 \pmod{2^\tau}$. Let $\mathbf{v} - \mathbf{v}' = (w_0, w_1, \dots, w_{\frac{d}{2}-1})$. We prove by induction that $w_j = 0$ for all $0 \leq j < \frac{d}{2}$.

Note that $\mathbf{W}_d$ is lower triangular, with the j -th diagonal entry given by $\phi^{w_H(j)}$. For $j = 0$, the condition $\mathbf{W}_d \cdot (\mathbf{v} - \mathbf{v}') = 0 \pmod{2^\tau}$ implies $w_0 = 0 \pmod{2^\tau}$. Write $w_0 = a + b\mathbf{i}$ with $-2^\tau < a, b < 2^\tau$ (since $\mathbf{v}, \mathbf{v}' \in \prod_{0 \leq j < n/2} \mathcal{S}_{2\tau - w_H(j)}$). It follows that $a = b = 0$. Now assume $w_0 = \dots = w_{j-1} = 0$; we show $w_j = 0$. Because $\mathbf{W}_d$ is lower triangular, the condition $\mathbf{W}_d \cdot (\mathbf{v} - \mathbf{v}') = 0 \pmod{2^\tau}$ gives $w_j \cdot \phi^{w_H(j)} = 0 \pmod{2^\tau}$. If $w_H(j)$ is even, then $\phi^{w_H(j)} = (\phi^2)^{w_H(j)/2} = (-2\mathbf{i})^{w_H(j)/2}$, so $w_j \cdot \phi^{w_H(j)} = 0 \pmod{2^\tau}$ implies $w_j \cdot 2^{w_H(j)/2} = 0 \pmod{2^\tau}$. Writing $w_j = a + b\mathbf{i}$ with $-2^{\tau - w_H(j)/2} < a, b < 2^{\tau - w_H(j)/2}$, we get $a = b = 0 \pmod{2^{\tau - w_H(j)/2}}$, hence $a = b = 0$. If $w_H(j)$ is odd, a similar argument shows $w_j \cdot \phi \cdot 2^{(w_H(j)-1)/2} = 0 \pmod{2^\tau}$. Write $w_j = a + b\mathbf{i}$ where $|a| < 2^{\tau - (w_H(j)-1)/2}$ and $|b| < 2^{\tau - (w_H(j)+1)/2}$. Then $a - b = a + b = 0 \pmod{2^{\tau - (w_H(j)-1)/2}}$. From $b = \frac{1}{2}((a+b) - (a-b))$, we obtain $b = 0 \pmod{2^{\tau - (w_H(j)-1)/2 - 1}}$, so $b = 0$. Consequently $a = 0 \pmod{2^{\tau - (w_H(j)-1)/2}}$, implying $a = 0$.

Putting the above deductions together, we conclude that **Label** is injective. It remains to show that **Delabel** is the inverse of **Label**.

Note that the computation in line 11 of Algorithm 3 cannot be directly inverted due to the modulo operation. To address this, **Delabel** adjusts the entries of the vector $\mathbf{w} = (v_0, \dots, v_{\frac{d}{2}-1})$ so that each v_j belongs to $\mathcal{S}_{2\tau - w_H(j)}$ (line 7 in Algorithm 5). We first show that this adjustment ensures $\phi^{2\tau - w_H(j)} \mid (v_j - v'_j)$. Observe that $v_j - v'_j = (a + b\mathbf{i}) - (a' + b'\mathbf{i}) = (a - a' - (b - b')) + (b - b')\phi$. Since $\phi^2 \mid 2$ and by the definitions of a' and b' , we obtain $\phi^{2\tau - w_H(j)} \mid (v_j - v'_j)$.

Now let $\mathbf{v} \in \prod_{0 \leq j < d/2} \mathcal{S}_{2\tau - w_H(j)}$ and set $\mathbf{w} = \lfloor \mathbf{W}_d \cdot \mathbf{v} \rfloor_{2^\tau}$. Define $\mathbf{W}_d^{-1} \cdot \mathbf{w} = (v_0, \dots, v_{\frac{d}{2}-1})$ and let $\mathbf{v}' = (v'_0, \dots, v'_{\frac{d}{2}-1})$ be the unique vector in $\prod_{0 \leq j < d/2} \mathcal{S}_{2\tau - w_H(j)}$ such that $\phi^{2\tau - w_H(j)} \mid (v_j - v'_j)$.

To prove that `Delabel` inverts `Label`, it suffices to show $\mathbf{v} = \mathbf{v}'$. Let $\boldsymbol{\delta} = (\delta_0, \dots, \delta_{\frac{d}{2}-1}) = \mathbf{W}_d^{-1} \mathbf{w} - \mathbf{v}'$. Then $\phi^{2\tau - w_H(j)} \mid \delta_j$ and

$$\mathbf{W}_d \cdot (\boldsymbol{\delta} + \mathbf{v}') = \mathbf{W}_d \cdot \mathbf{v} \pmod{2^\tau}. \quad (7)$$

Because each entry of the j -th column of $\mathbf{W}_d$ is divisible by $\phi^{w_H(j)}$, every component of $\mathbf{W}_d \cdot \boldsymbol{\delta}$ is divisible by $\phi^{2\tau}$. Hence $\mathbf{W}_d \cdot \boldsymbol{\delta} = 0 \pmod{2^\tau}$, and (7) implies $\mathbf{W}_d \cdot \mathbf{v}' = \mathbf{W}_d \cdot \mathbf{v} \pmod{2^\tau}$. By the same reasoning as in Lemma 2, we conclude $\mathbf{v} = \mathbf{v}'$, which completes the proof. $\square$

Using RBW_d as the Coding Lattice. This case covers the cases of Scloud⁺-256 and Scloud⁺-512, of which rotated Barnes-Wall lattices are adopted as the coding lattice.

For $d = 2^k \geq 4$, let the coding lattice $\mathcal{L}_c = \text{RBW}_d$ and the shaping lattice $\mathcal{L}_s = 2^\tau \cdot \mathbb{Z}[\mathbf{i}]^{d/2}$, where $\tau \geq \lceil \frac{k}{2} \rceil$ is a positive integer. Then by Lemma 1, it has $\mathcal{L}_s \subseteq 2^{\lceil \frac{k}{2} \rceil} \cdot \mathbb{Z}[\mathbf{i}]^{d/2} \subseteq \mathcal{L}_c$. Denote $\mathcal{C}$ to be the lattice code defined based on the nested lattices $\mathcal{L}_s \subseteq \mathcal{L}_c$, where each vector of $\mathcal{C}$ is required to be within the region $\mathcal{P} = \{(a_1 + b_1 \mathbf{i}, a_2 + b_2 \mathbf{i}, \dots, a_{d/2} + b_{d/2} \mathbf{i}) \mid 0 \leq a_j, b_j < 2^\tau \text{ for all } 1 \leq j \leq d/2\}$, i.e., the fundamental parallelepiped with respect to the standard basis of $\mathcal{L}_s = 2^\tau \cdot \mathbb{Z}[\mathbf{i}]^{d/2}$. By Lemma 1 it has

$$|\mathcal{C}| = \frac{\det(\mathcal{L}_s)}{\det(\mathcal{L}_c)} = \frac{2^{\tau d}}{2^{\frac{d}{4}(k+1)}} = 2^{\tau d - \frac{d}{4}(k+1)}. \quad (8)$$

A lemma analogous to Lemma 2 holds for the functions `Label` and `Delabel` defined in Algorithm 14 and Algorithm 16 for the rotated Barnes-Wall lattice RBW_d . The proof is similar and therefore omitted.

3.1.4 Decryption Failure Rate Analysis.

During decryption, we have

$$\mathbf{D} = \mathbf{C}_2 - \mathbf{C}_1 \mathbf{S} = \mathbf{S}' \mathbf{E} + \mathbf{E}_2 - \mathbf{E}_1 \mathbf{S} + \text{MsgEnc}(\mathbf{m}) \pmod{q}.$$

Define $\mathbf{E}_{\text{total}} := \mathbf{S}' \mathbf{E} + \mathbf{E}_2 - \mathbf{E}_1 \mathbf{S}$. Then decryption is correct if

$$\text{MsgDec}(\text{MsgEnc}(\mathbf{m}) + \mathbf{E}_{\text{total}}) = \mathbf{m}.$$

According to Algorithm 2 and Algorithm 4, $\mathbf{m}$ is partitioned into subvectors $\mathbf{m}_j$ for $1 \leq j \leq \sigma$. Consequently, $\text{MsgEnc}(\mathbf{m}) + \mathbf{E}_{\text{total}}$ can be truncated and partitioned into vectors $\frac{q}{g} \cdot \mathbf{w}_j + \boldsymbol{\epsilon}_j \in \mathbb{Z}[\mathbf{i}]^{d/2}$, where $\mathbf{w}_j$ is the output of $\text{MsgEnc}(\mathbf{m}_j)$ and $\boldsymbol{\epsilon}_j \in \mathbb{Z}[\mathbf{i}]^{d/2}$ corresponds to components of $\mathbf{E}_{\text{total}}$. By Lemma 2, we have $\text{MsgDec}(\text{MsgEnc}(\mathbf{m}) + \mathbf{E}_{\text{total}}) = \mathbf{m}$ if the BDD algorithm, given input $\mathbf{w}_j + \frac{q}{g} \cdot \boldsymbol{\epsilon}_j$, outputs $\mathbf{w}_j$ for all $1 \leq j \leq \sigma$. Since Algorithm 6 guarantees to return a lattice vector $\mathbf{y}$ satisfying $\|\mathbf{t} - \mathbf{y}\| < \lambda_1/2$ for a given input target vector $\mathbf{t}$ whose distance to BW_d (or RBW_d) is less than $\lambda_1/2$. Thus the decryption is correct when $\left\| \frac{q}{g} \cdot \boldsymbol{\epsilon}_j \right\| \leq r$ for all j , where r is the packing radius, i.e, $r = \sqrt{d/8}$ for BW_d and $r = \sqrt{d/4}$ for RBW_d .

It follows that

$$\Pr[\text{MsgDec}(\text{MsgEnc}(\mathbf{m}) + \mathbf{E}_{\text{total}}) = \mathbf{m}] = \Pr\left[\|\boldsymbol{\epsilon}_j\| \leq \frac{q}{g} \cdot r \text{ for all } j\right]. \quad (9)$$

Next, we analyze the distribution of the error matrix $\mathbf{E}_{\text{total}}$ to approximate (9). Since each component of $\mathbf{S}$ and $\mathbf{S}'$ is sampled independently from the Bernoulli-Difference distribution with parameter ρ_s , and each component of $\mathbf{E}$, $\mathbf{E}_1$, $\mathbf{E}_2$ is sampled from the Bernoulli-Difference distribution with parameter ρ_e , we find that each component of $\mathbf{E}_{\text{total}}$ follows a distribution with mean 0 and variance

$$\begin{aligned} \sigma_{\text{total}}^2 &= m \cdot (2\rho_s(1 - \rho_s)) \cdot (2\rho_e(1 - \rho_e)) + n \cdot (2\rho_e(1 - \rho_e)) \cdot (2\rho_s(1 - \rho_s)) + (2\rho_e(1 - \rho_e)) \\ &= 4(m + n)\rho_s\rho_e(1 - \rho_s)(1 - \rho_e) + 2\rho_e(1 - \rho_e). \end{aligned}$$

As is common in other schemes, we approximate the right-hand side of (9) using the Gaussian distribution. Assuming independence of the components of $\mathbf{E}_{\text{total}}$, we obtain, for each j ,

$$\Pr\left[\|\boldsymbol{\epsilon}_j\| \leq \frac{rq}{g}\right] \approx \sum_{\mathbf{z} \in \mathbb{Z}^d, \|\mathbf{z}\| \leq \frac{rq}{g}} \prod_{i=1}^d \frac{g\sigma_{\text{total}}(z_i)}{g\sigma_{\text{total}}(\mathbb{Z})}$$

$$\begin{aligned}
&\approx \int_{\mathbf{x} \in \mathbb{R}^d, \|\mathbf{x}\| \leq \frac{rq}{g}} \frac{1}{(2\pi\sigma_{\text{total}}^2)^{d/2}} e^{-\|\mathbf{x}\|^2/(2\sigma_{\text{total}}^2)} d\mathbf{x} \\
&= 1 - \Gamma\left(d/2, \left(\frac{rq}{\sqrt{2}g\sigma_{\text{total}}}\right)^2\right) / \Gamma(d/2, 0),
\end{aligned}$$

where $g_\sigma(z) = \frac{1}{\sqrt{2\pi}\sigma} e^{-z^2/(2\sigma^2)}$ is the Gaussian function, and $\Gamma(s, x) = \int_x^\infty t^{s-1} e^{-t} dt$ is the incomplete Gamma function. The second approximation treats the discrete Gaussian distribution as continuous. Thus, the decryption failure rate of the PKE scheme can be approximated as

$$\begin{aligned}
\text{DFR} &= 1 - \Pr\left[\|\epsilon_j\| \leq \frac{rq}{g} \text{ for all } j\right] \approx \sigma \cdot \left(1 - \Pr\left[\|\epsilon_j\| \leq \frac{rq}{g}\right]\right) \\
&\approx \sigma \cdot \Gamma\left(d/2, \left(\frac{rq}{\sqrt{2}g\sigma_{\text{total}}}\right)^2\right) / \Gamma(d/2, 0).
\end{aligned} \tag{10}$$

For the parameters selected in Table 2 and Table 3, one can verify that the right-hand side of (10) is less than $2^{-\lambda}$ with some margin for the target security levels $\lambda = 128, 192, 256, 384, 512$, respectively. The estimated decryption failure rates computed from Equation 10 are listed in Table 16.

Table 16. Decryption failure rates of Scloud⁺.

	Scloud ⁺ -128	Scloud ⁺ -192	Scloud ⁺ -256	Scloud ⁺ -384	Scloud ⁺ -512
DFR	$2^{-138.5}$	$2^{-220.9}$	$2^{-271.8}$	$2^{-457.7}$	$2^{-552.6}$

4 Security Proofs

This section establishes the security of the proposed schemes based on the LWE problem. We begin by recalling some fundamental cryptographic definitions.

4.1 Cryptographic Definitions

Definition 6 (PKE). A public-key encryption (PKE) scheme is a tuple of algorithms (KeyGen, Enc, Dec) along with a message space $\mathcal{M}$.

- The probabilistic key generation algorithm KeyGen outputs a pair of public key and secret key (pk, sk) .
- The probabilistic encryption algorithm Enc takes as input pk and a message $m \in \mathcal{M}$, and outputs a ciphertext c .
- The deterministic decryption algorithm Dec takes as input sk and c , and outputs either a message $m' \in \mathcal{M}$ or a special error symbol $\perp \notin \mathcal{M}$.

A PKE scheme is δ -correct if

$$\mathbb{E} \left[\max_{m \in \mathcal{M}} \Pr[\text{Dec}(sk, \text{Enc}(pk, m)) \neq m] \right] \leq \delta,$$

where the expectation is taken over $(pk, sk) \leftarrow \text{KeyGen}()$, and the probability is taken over the randomness of Enc. The PKE scheme in our construction is considered to satisfy IND-CPA security (indistinguishability under chosen plaintext attack). Specifically, the advantage of an adversary A is defined as

$$\text{Adv}_{\text{PKE}}^{\text{CPA}}(A) = \left| \Pr \left[b = b' \mid \begin{array}{l} (pk, sk) \leftarrow \text{KeyGen}(), (m_0, m_1, s) \leftarrow A(pk) \\ b \leftarrow \{0, 1\}, c^* \leftarrow \text{Enc}(pk, m_b), b' \leftarrow A(pk, c^*, s) \end{array} \right] - \frac{1}{2} \right|.$$

Definition 7 (KEM). A key encapsulation mechanism (KEM) is a tuple of algorithms (KeyGen, Encaps, Decaps) along with a key space $\mathcal{K}$.

- The probabilistic key generation algorithm KeyGen outputs a pair of public key and secret key (pk, sk) .
- The probabilistic encapsulation algorithm Encaps takes as input pk and outputs an ciphertext c and a shared secret $ss \in \mathcal{K}$.
- The deterministic decapsulation algorithm Decaps takes as input sk and c , and outputs a shared secret $ss' \in \mathcal{K}$.

A KEM is δ -correct if

$$\Pr[\text{Decaps}(sk, c) \neq ss \mid (c, ss) \leftarrow \text{Encaps}(pk)] \leq \delta,$$

where the probability is taken over $(pk, sk) \leftarrow \text{KeyGen}()$ and the randomness of *Encaps*. The KEM in our construction is considered to satisfy IND-CCA2 security (*indistinguishability under adaptive chosen ciphertext attack*). Specifically, the advantage of an adversary A is defined as

$$\text{Adv}_{\text{KEM}}^{\text{CCA}}(A) = \left| \Pr \left[b = b' \mid \begin{array}{l} (pk, sk) \leftarrow \text{KeyGen}(), b \leftarrow \{0, 1\} \\ (c^*, ss_0) \leftarrow \text{Encaps}(), ss_1 \leftarrow U(\mathcal{K}) \\ b' \leftarrow A^{\text{DECAPS}(\cdot)}(pk, ss_b, c^*) \end{array} \right] - \frac{1}{2} \right|,$$

where the DECAPS oracle is defined as $\text{DECAPS}(\cdot) := \text{Decaps}(sk, \cdot)$, and the adversary A is not allowed to make queries with input c^* .

4.2 Security of the PKE Scheme

The security of the proposed scheme relies on the LWE problem, which is defined as follows.

Definition 8 (LWE Distribution). Let n, q be positive integers, and let χ_e be a distribution over $\mathbb{Z}$. Given $\mathbf{s} \in \mathbb{Z}_q^n$, the LWE distribution $\mathcal{A}_{\mathbf{s}, \chi_e}$ outputs $(\mathbf{a}, \langle \mathbf{a}, \mathbf{s} \rangle + e \bmod q) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$, where $\mathbf{a} \leftarrow U(\mathbb{Z}_q^n)$ and $e \leftarrow \chi_e$.

Definition 9 (LWE Problem). Let n, m, q be positive integers, and let χ_e, χ_s be distributions over $\mathbb{Z}$ and $\mathbb{Z}^n$ respectively. The LWE problem is to distinguish between m samples $(\mathbf{a}_i, b_i) \leftarrow U(\mathbb{Z}_q^n \times \mathbb{Z}_q)$ and m samples $(\mathbf{a}_i, b_i) \leftarrow \mathcal{A}_{\mathbf{s}, \chi_e}$, where the secret vector $\mathbf{s} \leftarrow \chi_s$ is common to all samples. More formally, for an adversary A , we define

$$\text{Adv}_{n, m, q, \chi_e, \chi_s}^{\text{LWE}}(A) = \left| \Pr \left[A(\mathbf{A}, \mathbf{b}) = 1 \mid \begin{array}{l} \mathbf{A} \leftarrow U(\mathbb{Z}_q^{m \times n}), \mathbf{e} \leftarrow \chi_e^m \\ \mathbf{s} \leftarrow \chi_s, \mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e} \end{array} \right] - \Pr[A(\mathbf{A}, \mathbf{b}) = 1 \mid \mathbf{A} \leftarrow U(\mathbb{Z}_q^{m \times n}), \mathbf{b} \leftarrow U(\mathbb{Z}_q^m)] \right|.$$

In our scheme we choose χ_s, χ_e to be the bernoulli-difference distribution with parameter ρ_s, ρ_e . We denote the corresponding LWE problem to be $\text{Adv}_{n, m, q, \rho_s, \rho_e}^{\text{LWE}}(A)$. It is important to note that while the original LWE problem was established using the discrete Gaussian distribution [23, 6], the hardness of the LWE problem does not seem to be affected by the exact shape of the error distribution. The following theorem establishes the security of $\text{Scloud}^+.\text{PKE}$.

Theorem 1. $\text{Scloud}^+.\text{PKE}$ is IND-CPA secure, assuming that the LWE problem is hard and that the matrix $\mathbf{A}$ generated by *gen* is uniformly distributed over $\mathbb{Z}_q^{m \times n}$. Specifically, for any adversary C against the IND-CPA security of $\text{Scloud}^+.\text{PKE}$, there exist adversaries B_1, B_2 with running times approximately equal to that of C , such that

$$\text{Adv}_{\text{Scloud}^+.\text{PKE}}^{\text{CPA}}(C) \leq \bar{n} \cdot \text{Adv}_{n, m, q, \rho_s, \rho_e}^{\text{LWE}}(B_1) + \bar{m} \cdot \text{Adv}_{m + \bar{m}, n, q, \rho_s, \rho_e}^{\text{LWE}}(B_2).$$

Proof. We begin by considering the matrix-LWE problem where the secret and errors are also matrices (as in our PKE scheme). For an adversary A , we define

$$\text{Adv}_{n, m, \bar{n}, q, \rho_s, \rho_e}^{\text{matrix-LWE}}(A) = \left| \Pr \left[A(\mathbf{A}, \mathbf{B}) = 1 \mid \begin{array}{l} \mathbf{A} \leftarrow U(\mathbb{Z}_q^{m \times n}), \mathbf{E} \leftarrow \rho_e^{m \times \bar{n}} \\ \mathbf{S} \leftarrow \rho_s^{n \times \bar{n}}, \mathbf{B} = \mathbf{A}\mathbf{S} + \mathbf{E} \end{array} \right] \right|$$

$$- \Pr[\mathbf{A}(\mathbf{A}, \mathbf{B}) = 1 \mid \mathbf{A} \leftarrow U(\mathbb{Z}_q^{m \times n}), \mathbf{B} \leftarrow U(\mathbb{Z}_q^{m \times \bar{n}})] .$$

Using a standard hybrid argument [24], it can be shown that there exists an adversary $\mathbf{A}'$ with approximately the same running time as $\mathbf{A}$, such that:

$$\mathbf{Adv}_{n, m, q, \rho_s, \rho_e}^{\text{LWE}}(\mathbf{A}') \geq \frac{1}{\bar{n}} \mathbf{Adv}_{n, m, \bar{n}, q, \rho_s, \rho_e}^{\text{matrix-LWE}}(\mathbf{A}). \quad (11)$$

Now, let $\mathbf{C}$ be executed in the IND-CPA security game G_0 , where

$$\mathbf{Adv}_{\text{Scloud}^+. \text{PKE}}^{\text{CPA}}(\mathbf{C}) = |\Pr[b = b' \text{ in game } G_0] - 1/2|.$$

Define G_1 as the game where the matrix $\mathbf{B}$ in key generation is drawn from $U(\mathbb{Z}_q^{m \times \bar{n}})$ rather than generated via $\mathbf{B} = \mathbf{A} \cdot \mathbf{S} + \mathbf{E}$. Then, there exists an adversary $\mathbf{B}'_1$ with a running time comparable to that of $\mathbf{C}$ such that

$$|\Pr[b = b' \text{ in game } G_0] - \Pr[b = b' \text{ in game } G_1]| \leq \mathbf{Adv}_{n, m, \bar{n}, q, \rho_s, \rho_e}^{\text{matrix-LWE}}(\mathbf{B}'_1).$$

Define game G_2 where the matrix $(\mathbf{C}_1 = \mathbf{S}' \cdot \mathbf{A} + \mathbf{E}_1, \mathbf{C}'_2 = \mathbf{S}' \cdot \mathbf{B} + \mathbf{E}_2)$ used to generate the challenge ciphertext is drawn from $U(\mathbb{Z}_q^{\bar{m} \times n} \times \mathbb{Z}_q^{\bar{m} \times \bar{n}})$. Similarly, there exists an adversary $\mathbf{B}'_2$ with a running time similar to that of $\mathbf{C}$ such that

$$|\Pr[b = b' \text{ in game } G_1] - \Pr[b = b' \text{ in game } G_2]| \leq \mathbf{Adv}_{m + \bar{m}, n, \bar{m}, q, \rho_s, \rho_e}^{\text{matrix-LWE}}(\mathbf{B}'_2).$$

In game G_2 , the ciphertext $\mathbf{C}_2 = \mathbf{C}'_2 + \mathbf{M}$ is uniformly distributed over $\mathbb{Z}_q^{\bar{m} \times \bar{n}}$ and is independent of the message $\mathbf{m}$, implying that $\Pr[b = b' \text{ in game } G_2] = 1/2$. Thus, from the above analysis, we obtain

$$\mathbf{Adv}_{\text{Scloud}^+. \text{PKE}}^{\text{CPA}}(\mathbf{C}) \leq \mathbf{Adv}_{n, m, \bar{n}, q, \rho_s, \rho_e}^{\text{matrix-LWE}}(\mathbf{B}'_1) + \mathbf{Adv}_{m + \bar{m}, n, \bar{m}, q, \rho_s, \rho_e}^{\text{matrix-LWE}}(\mathbf{B}'_2).$$

The theorem follows directly by combining this result with (11). $\square$

4.3 Security of the KEM Scheme

It is evident that the failure probability δ of $\text{Scloud}^+. \text{KEM}$ is equal to the failure probability of $\text{Scloud}^+. \text{PKE}$. Note that $\mathbf{H}$, $\mathbf{G}$, and $\mathbf{KDF}$ involved in $\text{Scloud}^+. \text{KEM}$ can be modeled as independent random oracles, which is guaranteed by our choice of symmetric cryptographic primitives (see Section 2.1). Following the approach of [3], we can derive the following security bounds, for which the proof is omitted in this work.

Theorem 2. *Suppose that $\text{Scloud}^+. \text{PKE}$ is a δ -correct PKE with message space $\mathcal{M}$. Then for any classical adversary $\mathbf{A}$ against $\text{Scloud}^+. \text{KEM}$ that makes at most q_{RO} random oracle queries, there exists a classical adversary $\mathbf{B}$ against $\text{Scloud}^+. \text{PKE}$, whose running time is approximately the same as that of $\mathbf{A}$, such that*

$$\mathbf{Adv}_{\text{Scloud}^+. \text{KEM}}^{\text{CCA}}(\mathbf{A}) \leq 3 \cdot \mathbf{Adv}_{\text{Scloud}^+. \text{PKE}}^{\text{CPA}}(\mathbf{B}) + \frac{3q_{RO}}{|\mathcal{M}|} + q_{RO} \cdot \delta.$$

For security in the *quantum* random oracle model, the approach proposed in [25,26] can be applied to obtain the following bound.

Theorem 3. *Suppose that $\text{Scloud}^+. \text{PKE}$ is a δ -correct PKE with message space $\mathcal{M}$. Then for any quantum adversary $\mathbf{A}$ against $\text{Scloud}^+. \text{KEM}$, making at most q_F quantum oracle queries to $\mathbf{KDF}$ and at most q_G quantum oracle queries to $\mathbf{G}$, there exists a quantum adversary $\mathbf{B}$ against $\text{Scloud}^+. \text{PKE}$, whose running time is approximately the same as that of $\mathbf{A}$, such that*

$$\mathbf{Adv}_{\text{Scloud}^+. \text{KEM}}^{\text{CCA}}(\mathbf{A}) \leq 2\sqrt{q_{RO} \cdot \mathbf{Adv}_{\text{Scloud}^+. \text{PKE}}^{\text{CPA}}(\mathbf{B}) + \frac{2(q_{RO} + 1)^2}{|\mathcal{M}|}} + \frac{2q_F}{\sqrt{|\mathcal{M}|}} + 4q_G \cdot \sqrt{\delta},$$

where $q_{RO} = q_F + q_G$.

5 Security Against Known Attacks

This subsection provides a concrete security analysis for the parameter sets of Scloud⁺.

5.1 Lattice Reduction

For most known attacks on LWE problems, one first embeds the LWE instance into a suitable lattice and then relies on lattice reduction to recover a short vector or to construct a short distinguishing vector. This paradigm appears in primal attacks, dual attacks, and their hybrid variants. Consequently, algorithms that provide sufficiently short vectors in high-dimensional lattices are inevitably invoked as the dominant subroutines in concrete LWE security estimates.

Among lattice-reduction algorithms, BKZ [27] and its variants [28, 29], are the standard tools used in concrete analyses of LWE-based schemes. The BKZ block size b controls the quality of the reduced basis: larger blocks give shorter vectors, but they require solving harder local SVP instances. In a BKZ- b reduction, the algorithm repeatedly calls an SVP oracle on projected sublattices of dimension about b , and these SVP calls dominate the running time.

Concrete LWE security estimates mainly differ in how the local SVP oracle in BKZ is priced. Common choices include enumeration-based models [28, 30, 31], sieving-based reduction [32, 33, 34], including core-SVP models which a single SVP call is used to estimate the whole reduction cost [35, 36]; and progressive-BKZ detailed estimation as in [37, 38] which is usually called as "list decoding". Since these models can yield noticeably different concrete estimates for the same parameter set, we explicitly adopt the conservative core-SVP models as our main baseline for a clean and widely comparable analysis.

Core-SVP Hardness. The *core-SVP* model is typically used to obtain a conservative estimate of the BKZ algorithm's complexity, where $o(b)$ and $\text{poly}(b)$ factors are ignored, and only the core 2^{cb} complexity is considered. The best known constant c for classical algorithms is $c = \log_2(\sqrt{3}/2) \approx 0.292$, as derived from the sieve algorithm [32]. For quantum algorithms, the constant c is usually taken to be $c = \log_2(\sqrt{13}/9) \approx 0.265$ [33, 34]. Although a quantum random walk approach has been shown to provide an improved quantum algorithm for solving SVP with $c \approx 0.257$ [36], this method has not been widely adopted in the analysis of LWE-based schemes. Therefore, in the analysis of quantum security, we adhere to the common choice of $c \approx 0.265$.

5.2 Attacks for LWE.

We consider the standard attack families used in concrete LWE security estimation. The main class includes primal attack, dual attack, and their hybrid variants.

5.2.1 Primal Attack. For a given LWE instance $(\mathbf{A} \in \mathbb{Z}_q^{m \times n}, \mathbf{b} = \mathbf{As} + \mathbf{e} \in \mathbb{Z}_q^m)$, the primal attack embeds the instance into a q -ary lattice and solve a short-vector problem by lattice reduction. There are two kinds of models in solving this instance, primal-uSVP and primal-BDD. Intuitively, they describe the same geometric phenomenon from different angles: uSVP emphasizes uniqueness of a short vector, while BDD emphasizes successful nearest-plane/enum decoding to that vector.

Primal-uSVP. One builds an embedding lattice $\Lambda = \{\mathbf{x} \in \mathbb{Z}^{m+n+1} \mid (\mathbf{A} \mid \mathbf{I}_m \mid -\mathbf{b})\mathbf{x} = 0 \pmod{q}\}$ then finds the unique shortest vector $(\mathbf{s}, \mathbf{e}, 1)$ using the BKZ algorithm.

Primal-BDD. The LWE instance $(\mathbf{A} \in \mathbb{Z}_q^{m \times n}, \mathbf{b} = \mathbf{As} + \mathbf{e} \in \mathbb{Z}_q^m)$ is a BDD instance for the lattice $\Lambda_q(\mathbf{A}) = \{\mathbf{x} \in \mathbb{Z}^m \mid \mathbf{x} = \mathbf{Az} \pmod{q}\}$, where the target is the embedded LWE vector $(\mathbf{b}, -\mathbf{s})$ and the decoding radius is determined by the error norm $\|\mathbf{e}\|$.

Primal-Hybrid. When the secret is sparse or ternary, the primal attack can be optimized by guessing part of the secret key which is called the primal-hybrid attack. It first guesses a subset of secret coordinates, then runs a primal-uSVP/BDD-style reduction on the residual lower-dimensional instance. The total cost is a trade-off between the guessing phase (typically combinatorial search or meet-in-the-middle style acceleration) and the reduced BKZ cost after dimension reduction; the optimal split is chosen by minimizing the combined runtime. We refer to [39, 40] for a detailed analysis of the primal-hybrid attack.

5.2.2 Dual Attack. Instead of directly recovering the secret, the dual attack constructs a short vector in a dual lattice and uses it to distinguish the LWE samples from uniform. In our analysis, we consider the following variants of the dual attack.

Dual attack (Plain). For a given LWE instance $(\mathbf{A} \in \mathbb{Z}_q^{m \times n}, \mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e} \in \mathbb{Z}_q^m)$, the dual attack first finds short vectors $\{\mathbf{x}_j, \mathbf{y}_j\}_{1 \leq j \leq N}$ in the lattice $\Lambda' = \{(\mathbf{x}, \mathbf{y}) \in \mathbb{Z}^{n+m} \mid \mathbf{x}^\top \mathbf{A} = \mathbf{y}^\top \pmod{q}\}$, and then tries to distinguish the samples $\{\mathbf{x}_j^\top \mathbf{b}\}_{1 \leq j \leq N}$ from samples drawn from $\mathbb{Z}_q$. We refer to [41] for a detailed analysis of the dual attack.

Dual-Hybrid (General). This model is especially relevant for binary, sparse, or ternary secrets, where guessing part of the secret can reduce the lattice-reduction dimension enough to offset the guessing cost [41, 42]. It first uses the dual vector to reduce the original instance to a smaller residual LWE instance on only ζ secret coordinates, and then solves this residual instance by exhaustive search or another solver. The total cost is a trade-off between the guessing phase and the reduced BKZ cost after dimension reduction; the optimal split is chosen by minimizing the combined runtime. We refer to [41] for a detailed analysis of the dual-hybrid attack.

Dual-Hybrid (FFT and MATZOV). In the hybrid dual attack variants, the FFT style distinguishers are sometimes applied, and refine the distinguishing phase by guessing coordinates modulo a small modulus and using fast transforms. In the MATZOV model, it combines optimized dual-vector sampling, guessing, FFT/list-decoding style subroutines, and a detailed search over parameters such as the BKZ block size, the number of guessed coordinates, the FFT dimension, and the guessing modulus. These models are often more aggressive and depend on additional heuristic and implementation-level assumptions [43, 38, 44], which have not been widely accepted.

5.2.3 Concrete hardness of our parameter. We provide a concrete security analysis of our scheme using the primal, dual, and its related hybrid variants. The primal-type results are summarized in Table 17, and the dual-type results are summarized in Table 18.

According to the reductions established in Section 4, it suffices to consider the LWE instances with parameters $(n, m, q, \rho_s, \rho_e)$ to evaluate the security of the scheme.

Table 17. The estimated bits of security of Scloud⁺ under primal-type attack models.

Parameter	Model	Classical	Quantum
Scloud+ 128	primal-uSVP	142.08	128.85
	primal-BDD	142.94	129.72
	primal-hybrid	142.94	129.72
Scloud+ 192	primal-uSVP	211.70	192.12
	primal-BDD	212.52	192.96
	primal-hybrid	212.52	192.96
Scloud+ 256	primal-uSVP	291.42	264.47
	primal-BDD	292.15	265.23
	primal-hybrid	292.15	265.23
Scloud+ 384	primal-uSVP	426.49	386.96
	primal-BDD	427.41	387.88
	primal-hybrid	427.41	387.88
Scloud+ 512	primal-uSVP	593.51	538.54
	primal-BDD	594.25	539.30
	primal-hybrid	590.84	539.30

Table 18. The estimated bits of security of Scloud⁺ under dual-type attack models.

Parameter	Model	Classical	Quantum
Scloud+ 128	dual	148.21	134.42
	dual-hybrid	145.18	134.38
	dual-hybrid-FFT	145.18	134.38
	dual-hybrid(MATZOV)	134.22	123.72
Scloud+ 192	dual	220.17	199.81
	dual-hybrid	213.95	197.82
	dual-hybrid-FFT	213.95	197.82
	dual-hybrid(MATZOV)	198.24	181.88
Scloud+ 256	dual	301.93	274.01
	dual-hybrid	284.21	264.47
	dual-hybrid-FFT	284.21	264.47
	dual-hybrid(MATZOV)	266.03	244.37
Scloud+ 384	dual	440.80	399.94
	dual-hybrid	410.34	381.85
	dual-hybrid-FFT	410.34	381.85
	dual-hybrid(MATZOV)	387.38	355.99
Scloud+ 512	dual	611.62	554.97
	dual-hybrid	516.16	486.01
	dual-hybrid-FFT	516.16	486.01
	dual-hybrid(MATZOV)	535.13	492.80

Appendices

A Message Encoding and Decoding for RBW_d

The encoding and decoding procedures of ScCloud^+ for RBW_d are implemented by the functions described in [Algorithm 13](#) and [Algorithm 15](#), respectively. Furthermore, [Algorithm 13](#) calls the labeling function `Label` outlined in [Algorithm 14](#), while `MsgDec` calls the delabeling algorithm `Delabel` outlined in [Algorithm 16](#) and the BDD algorithm outlined in [Algorithm 17](#).

Algorithm 13 The `MsgEnc` Function for RBW_d

Input: Message $\mathbf{m} \in \{0, 1\}^{l_m}$, power-of-two integer $q = 2^t$, matrix dimensions $\bar{m}, \bar{n}$

Input: Lattice RBW_d , power-of-two integer $g = 2^\tau$, repetition number σ

Output: Matrix $\mathbf{M} \in \mathbb{Z}_q^{\bar{m} \times \bar{n}}$

- 1: Divide $\mathbf{m}$ into σ subvectors $\mathbf{m} = \mathbf{m}_1 || \mathbf{m}_2 || \dots || \mathbf{m}_\sigma$, where $\mathbf{m}_j \in \{0, 1\}^{l_m/\sigma}$
 - 2: **for** $j = 1$ to σ **do**
 - 3: Invoke `Label` with input RBW_d , $g = 2^\tau$ and $\mathbf{m}_j$ to obtain a lattice vector $\mathbf{w}_j \in \mathbb{Z}[\mathbf{i}]^{d/2}$
 - 4: Write $\mathbf{w}_j$ as $\mathbf{w}_j = \mathbf{u}_j + \mathbf{v}_j \mathbf{i}$, where $\mathbf{u}_j, \mathbf{v}_j \in \mathbb{Z}^{d/2}$
 - 5: **end for**
 - 6: $\mathbf{x} \leftarrow (\mathbf{u}_1, \mathbf{v}_1, \mathbf{u}_2, \mathbf{v}_2, \dots) \in \mathbb{Z}^{d \cdot \sigma}$
 - 7: Append zeros to $\mathbf{x}$ to extend it to a vector $\mathbf{x}' \in \mathbb{Z}^{\bar{m}\bar{n}}$
 - 8: $\mathbf{y} = (y_1, \dots, y_{\bar{m}\bar{n}}) \leftarrow (q/g) \cdot \mathbf{x}'$
 - 9: Assemble matrix $\mathbf{M}$ by setting the (i, j) -th entry to $y_{\bar{n}(i-1)+j}$ for $1 \leq i \leq \bar{m}$ and $1 \leq j \leq \bar{n}$
 - 10: **return** $\mathbf{M}$
-

Algorithm 14 The `Label` Function for RBW_d

Input: Lattice RBW_d with $d = 2^k$, $g = 2^\tau$

Input: $\mathbf{m} \in \{0, 1\}^*$ of length at most $\tau d - \frac{d}{4}(k+1)$ for RBW_d

Output: A lattice vector $\mathbf{x} \in \text{RBW}_d$

- 1: Pad $\mathbf{m}$ with zeros to obtain $\mathbf{m}' = \mathbf{m} || \mathbf{0} \in \{0, 1\}^{\tau d - \frac{d}{4}(k+1)}$
 - 2: Write $\mathbf{m}' = (\mathbf{u}_0, \mathbf{u}_1, \dots, \mathbf{u}_{\frac{d}{2}-1})$ such that $\mathbf{u}_j \in \{0, 1\}^{2\tau - w_H(j) - 1}$
 - 3: **for** j from 0 to $\frac{d}{2} - 1$ **do**
 - 4: $v_j \leftarrow f_{2\tau - w_H(j) - 1}(\mathbf{u}_j) \in \mathbb{Z}[\mathbf{i}]$
 - 5: **end for**
 - 6: $\mathbf{v} \leftarrow (v_0, \dots, v_{\frac{d}{2}-1})$
 - 7: **for** l from 1 to $k - 1$ **do**
 - 8: Write $\mathbf{v} = (\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_{\frac{d}{2^l}})$, where $\mathbf{w}_j \in \mathbb{Z}[\mathbf{i}]^{2^{l-1}}$
 - 9: Update $\mathbf{v} \leftarrow (\mathbf{w}_1, \mathbf{w}_1 + \phi \mathbf{w}_2, \mathbf{w}_3, \mathbf{w}_3 + \phi \mathbf{w}_4, \dots, \mathbf{w}_{\frac{d}{2^l-1}}, \mathbf{w}_{\frac{d}{2^l-1}} + \phi \mathbf{w}_{\frac{d}{2^l}})$, where $\phi = 1 + \mathbf{i}$
 - 10: **end for**
 - 11: $\mathbf{w} \leftarrow \phi \cdot [\mathbf{v}]_{2^\tau}$
 - 12: **return** $\mathbf{w}$
-

Algorithm 15 The `MsgDec` Function for RBW_d **Input:** Matrix $\mathbf{M} \in \mathbb{Z}_q^{\bar{m} \times \bar{n}}$, message length l_m **Input:** Lattice RBW_d , repetition number σ , and power-of-two integers $q = 2^t$, $g = 2^\tau$ **Output:** Message $\mathbf{m} \in \{0, 1\}^{l_m}$

- 1: $\mathbf{y} \leftarrow (y_1, y_2, \dots, y_{\bar{m}\bar{n}})$ such that $y_{\bar{n}(i-1)+j}$ is the (i, j) -th element of $\mathbf{M}$
- 2: Truncate $\mathbf{y}$ to its first $d \cdot \sigma$ entries to form vector $\mathbf{x} \in \mathbb{Z}^{d \cdot \sigma}$
- 3: $\mathbf{x} \leftarrow \frac{2^\tau}{q} \cdot \mathbf{x}$
- 4: Partition $\mathbf{x}$ as $\mathbf{x} = (\mathbf{u}_1, \mathbf{v}_1, \mathbf{u}_2, \mathbf{v}_2, \dots)$ such that $\mathbf{u}_j, \mathbf{v}_j \in \mathbb{Z}^{d/2}$
- 5: **for** $j = 1$ to σ **do**
- 6: Invoke BDD on input target $\mathbf{u}_j + \mathbf{v}_j \mathbf{i}$ to obtain lattice vector $\mathbf{w}_j$ in RBW_d
- 7: Invoke `Delabel` with input RBW_d , $g = 2^\tau$ and $[\mathbf{w}_j]_{2^\tau}$ to obtain $\mathbf{m}_j \in \{0, 1\}^{l_m/\sigma}$
- 8: **end for**
- 9: **return** $\mathbf{m} = \mathbf{m}_1 || \dots || \mathbf{m}_\sigma$

The only difference between [Algorithm 13](#) and [Algorithm 2](#) lies in line 3, namely the invocation of the labeling function. The difference between [Algorithm 14](#) and [Algorithm 3](#) is in the length of the input message: $\tau d - \frac{d}{4}(k+1)$ for RBW_d versus $\tau d - \frac{d}{4}(k-1)$ for BW_d . This disparity follows from the difference in volume between RBW_d and BW_d , as explained in [Section 3](#).

Algorithm 16 The `Delabel` Function for RBW_d **Input:** Lattice RBW_d with $d = 2^k$, $g = 2^\tau$ **Input:** A lattice vector $\mathbf{x} \in \text{RBW}_d$ **Output:** $\mathbf{m} \in \{0, 1\}^*$

- 1: $\mathbf{w} \leftarrow \phi^{-1} \mathbf{w}$ ▷ map the rotated codeword back to BW_d
- 2: **for** l from $k-1$ to 1 **do**
- 3: Write $\mathbf{w} = (\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_{n/2^l})$, where $\mathbf{w}_j \in \mathbb{Z}[\mathbf{i}]^{2^{l-1}}$ for $1 \leq j \leq d/2^l$
- 4: Update $\mathbf{w} \leftarrow (\mathbf{w}_1, \phi^{-1}(\mathbf{w}_2 - \mathbf{w}_1), \mathbf{w}_3, \phi^{-1}(\mathbf{w}_4 - \mathbf{w}_3), \dots)$
- 5: **end for**
- 6: Write $\mathbf{w} = (a_0, \dots, a_{\frac{d}{2}-1}) + (b_0, \dots, b_{\frac{d}{2}-1}) \cdot \mathbf{i}$
- 7: **for** j from 0 to $\frac{d}{2} - 1$ **do**
- 8: $b'_j \leftarrow [b_j]_{2^{\tau - \lceil (w_H(j)+1)/2 \rceil}}$, $a'_j \leftarrow [a_j - (b_j - b'_j)]_{2^{\tau - \lfloor (w_H(j)+1)/2 \rfloor}}$, $\mathbf{u}_j \leftarrow f_{2^{\tau - w_H(j)-1}}^{-1}(a'_j + b'_j \mathbf{i})$
- 9: **end for**
- 10: **return** $\mathbf{m} = \mathbf{u}_0 || \mathbf{u}_1 || \dots || \mathbf{u}_{\frac{d}{2}-1}$

The difference between [Algorithm 13](#) and [Algorithm 2](#) is the use of the rotated labeling function. The function [Algorithm 14](#) uses shorter component labels, of total length $\tau d - \frac{d}{4}(k+1)$ for RBW_d instead of $\tau d - \frac{d}{4}(k-1)$ for BW_d , and then applies the public rotation by ϕ . This disparity follows from the difference in volume between RBW_d and BW_d , as explained in [Section 3](#).

The difference between [Algorithm 15](#) and [Algorithm 4](#) is that decoding is performed for the rotated lattice and the result is passed to the rotated delabeling function. The difference between [Algorithm 16](#) and [Algorithm 5](#) is the initial multiplication by ϕ^{-1} , which removes the public rotation, and the computation of the inverse of the f function with the shorter component lengths. The difference between [Algorithm 17](#) and [Algorithm 6](#) lies in the recursions and in the different base cases for RBW_2 versus BW_2 . We note that for the base case $\text{RBW}_2 = \phi \cdot \text{BW}_2 = \phi \cdot \mathbb{Z}[\mathbf{i}]$, BDD can be viewed as a trivial instance of Babai's nearest plane algorithm.

Algorithm 17 BDD Algorithm for RBW_d **Input:** A target vector $\mathbf{t} \in \mathbb{C}^{d/2}$ and lattice RBW_d **Output:** A lattice vector $\mathbf{y}$

```

1: if  $d = 2$  then
2:   return  $\phi \cdot \lfloor \phi^{-1} \cdot \mathbf{t} \rfloor$ , where  $\phi = 1 + i$ 
3: else
4:   Write  $\mathbf{t} = (\mathbf{t}_1, \mathbf{t}_2)$  such that  $\mathbf{t}_1, \mathbf{t}_2 \in \mathbb{C}^{d/4}$ 
5:   Compute  $\mathbf{y}_1 = \text{BDD}(\mathbf{t}_1, \text{RBW}_{d/2})$ ,  $\mathbf{y}_2 = \text{BDD}(\mathbf{t}_2, \text{RBW}_{d/2})$ 
6:   Compute  $\mathbf{z}_1 = \text{BDD}(\phi^{-1}(\mathbf{t}_2 - \mathbf{y}_1), \text{RBW}_{d/2})$ ,  $\mathbf{z}_2 = \text{BDD}(\phi^{-1}(\mathbf{t}_1 - \mathbf{y}_2), \text{RBW}_{d/2})$ 
7:   Compute  $\mathbf{x} = (\mathbf{y}_1, \mathbf{y}_1 + \phi \mathbf{z}_1)$ ,  $\mathbf{x}' = (\mathbf{y}_2 + \phi \mathbf{z}_2, \mathbf{y}_2)$ 
8:   if  $\|\mathbf{x} - \mathbf{t}\| < \|\mathbf{x}' - \mathbf{t}\|$  then
9:     return  $\mathbf{x}$ 
10:  else
11:    return  $\mathbf{x}'$ 
12:  end if
13: end if

```

The difference between [Algorithm 15](#) and [Algorithm 4](#) is in line 6, namely the invocation of the delabeling function. The difference between [Algorithm 16](#) and [Algorithm 5](#) is in line 7, which involves the computation of the inverse of the f function, which is a result of the difference in the length of each $\mathbf{u}_j$. The difference between [Algorithm 17](#) and [Algorithm 6](#) lies in the recursions on lines 56, and in line 2 for the different base cases for RBW_2 versus BW_2 . We note that for the base case $\text{RBW}_2 = \phi \cdot \text{BW}_2 = \phi \cdot \mathbb{Z}[i]$, BDD can be viewed as a trivial instance of Babai's nearest plane algorithm.

B Integer Versions of the MsgDec and BDD Functions.

[Algorithm 18](#) presents the integer version of the message decoding function from [Algorithm 4](#). The main differences are in line 3 and line 6, where scaling by 2^t is introduced for exact arithmetic, achieved by invoking the integer version of the BDD algorithm ([Algorithm 19](#)). In the integer BDD, the target $\mathbf{t}$ is given as a vector in $\mathbb{Z}[i]^{d/2}$ with an implicit denominator 2^s , and the output is a lattice vector scaled by 2^s . Consequently, the base case ($d = 2$) returns $2^s \cdot \lfloor \mathbf{t}/2^s \rfloor$ instead of plain complex rounding. Each recursive call carries the current denominator as an extra argument. After the ϕ^{-1} step (multiplication by $1 - i$), the denominator is doubled to 2^{s+1} to keep all quantities integral. The resulting $\mathbf{z}_i$ are then halved (line 8) before being combined with $\mathbf{y}_i$ via ϕ , realigning the scales. Finally, the better candidate is selected using the squared Euclidean distance $\|\cdot\|^2$, which is exact in integers and avoids the floatingpoint square root used in the original algorithm.

Algorithm 18 The MsgDec-int Function (Integer Version)**Input:** Matrix $\mathbf{M} \in \mathbb{Z}_q^{\bar{m} \times \bar{n}}$, message length l_m **Input:** Lattice BW_d , repetition number σ , and power-of-two integers $q = 2^t$, $g = 2^\tau$ **Output:** Message $\mathbf{m} \in \{0, 1\}^{l_m}$

```

1:  $\mathbf{y} \leftarrow (y_1, y_2, \dots, y_{\bar{m}\bar{n}})$  such that  $y_{\bar{n}(i-1)+j}$  is the  $(i, j)$ -th element of  $\mathbf{M}$ 
2: Truncate  $\mathbf{y}$  to its first  $d \cdot \sigma$  entries to form  $\mathbf{x} \in \mathbb{Z}^{d \cdot \sigma}$ 
3: Partition  $\mathbf{x}$  as  $(\mathbf{u}_1, \mathbf{v}_1, \mathbf{u}_2, \mathbf{v}_2, \dots)$  such that  $\mathbf{u}_j, \mathbf{v}_j \in \mathbb{Z}^{d/2}$ 
4: for  $j = 1$  to  $\sigma$  do
5:   Invoke BDD-int on input  $\mathbf{u}_j + \mathbf{v}_j i$  with scaling factor  $2^{t-\tau}$  to obtain vector  $\mathbf{w}_j$  // returns vector scaled by  $2^{t-\tau}$ 
6:    $\mathbf{w}_j \leftarrow \mathbf{w}_j / 2^{t-\tau}$  // exact componentwise division, yields  $\mathbf{w}_j \in \mathbb{Z}[i]^{d/2}$ 
7:   Invoke Delabel with input  $\text{BW}_d$ ,  $g = 2^\tau$  and  $[\mathbf{w}_j]_{2^\tau}$  to obtain  $\mathbf{m}_j \in \{0, 1\}^{l_m/\sigma}$ 
8: end for
9: return  $\mathbf{m} = \mathbf{m}_1 || \dots || \mathbf{m}_\sigma$ 

```

Algorithm 19 BDD-int Algorithm for Barnes-Wall Lattices (Integer Version)**Input:** Target $\mathbf{t} \in \mathbb{Z}[i]^{d/2}$ with scaling factor 2^s , lattice BW_d **Output:** Lattice vector $\mathbf{y} \in 2^s \cdot \text{BW}_d$

```

1: if  $d = 2$  then
2:    $\mathbf{y} \leftarrow 2^s \cdot \lfloor \mathbf{t}/2^s \rfloor$  // round each component to nearest integer
3:   return  $\mathbf{y}$ 
4: else
5:   Write  $\mathbf{t} = (\mathbf{t}_1, \mathbf{t}_2)$  where  $\mathbf{t}_1, \mathbf{t}_2 \in \mathbb{Z}[i]^{d/4}$ 
6:    $\mathbf{y}_1 \leftarrow \text{BDD-int}(\mathbf{t}_1, 2^s, \text{BW}_{d/2})$ ,  $\mathbf{y}_2 \leftarrow \text{BDD-int}(\mathbf{t}_2, 2^s, \text{BW}_{d/2})$ 
7:    $\mathbf{z}_1 \leftarrow \text{BDD-int}((1-i)(\mathbf{t}_2 - \mathbf{y}_1), 2^{s+1}, \text{BW}_{d/2})$ ,  $\mathbf{z}_2 \leftarrow \text{BDD-int}((1-i)(\mathbf{t}_1 - \mathbf{y}_2), 2^{s+1}, \text{BW}_{d/2})$  // Apply  $\phi^{-1}$ : multiply by  $1-i$  and increase scaling to  $2^{s+1}$ 
8:    $\mathbf{z}'_1 \leftarrow \mathbf{z}_1/2$ ,  $\mathbf{z}'_2 \leftarrow \mathbf{z}_2/2$  // Align scales
9:   Compute  $\mathbf{x} = (\mathbf{y}_1, \mathbf{y}_1 + \phi \mathbf{z}'_1)$ ,  $\mathbf{x}' = (\mathbf{y}_2 + \phi \mathbf{z}'_2, \mathbf{y}_2)$ 
10:  if  $\|\mathbf{x} - \mathbf{t}\|^2 < \|\mathbf{x}' - \mathbf{t}\|^2$  then
11:    return  $\mathbf{x}$ 
12:  else
13:    return  $\mathbf{x}'$ 
14:  end if
15: end if

```

C Reference Implementation Results

This section presents results for the reference implementation.

C.1 AES-based Results (Reference Implementation).

Table 19, Table 20, and Table 21 present the reference performance of $\text{Scloud}^+.\text{KEM}$ (AES family) on `x86_64`, Linux AArch64, and macOS arm64, respectively.

Table 19. The `x86_64` reference performance of $\text{Scloud}^+.\text{KEM}$ with AES, measured in 10^3 cycles.

Level	KeyGen	Encaps	Decaps
128	17232.6	17583.7	17376.6
192	32296.0	32751.4	32728.3
256	65905.2	66739.3	66684.4
384	132107.5	134095.3	133867.1
512	265232.7	277131.6	267634.8

Table 20. The Linux AArch64 reference performance of $\text{Scloud}^+.\text{KEM}$ with AES, measured in microseconds.

Level	KeyGen	Encaps	Decaps
128	4257.2	4306.3	4240.4
192	7900.7	8071.9	8111.3
256	16128.1	16434.7	16509.1
384	32521.6	33053.8	33335.2
512	68265.7	68514.7	68979.2

Table 21. The macOS arm64 reference performance of Scloud⁺.KEM with AES , measured in microseconds.

Level	KeyGen	Encaps	Decaps
128	2038.9	2071.7	2075.1
192	3917.7	4033.1	4020.2
256	8333.9	8507.1	8533.3
384	16354.9	16682.8	16699.1
512	34312.6	34861.3	34969.5

C.2 SHAKE-based Results (Reference Implementation).

Table 22, Table 23, and Table 24 present the reference performance of Scloud⁺.KEM (SHAKE family) on x86_64, Linux AArch64, and macOS arm64, respectively.

Table 22. The x86_64 reference performance of Scloud⁺.KEM with SHAKE, measured in 10³ cycles.

Level	KeyGen	Encaps	Decaps
128	5832.1	5950.4	5883.9
192	11262.2	11718.5	11684.2
256	21305.9	22655.2	22589.8
384	45459.6	46334.1	46492.7
512	91428.6	90111.8	89160.9

Table 23. The Linux AArch64 reference performance of Scloud⁺.KEM with SHAKE, measured in microseconds.

Level	KeyGen	Encaps	Decaps
128	1386.0	1436.8	1424.3
192	2629.3	2967.0	3249.1
256	5142.4	5459.7	5745.2
384	10859.8	11461.4	11546.6
512	22662.8	22908.2	23021.0

Table 24. The macOS arm64 reference performance of Scloud⁺.KEM with SHAKE, measured in microseconds.

Level	KeyGen	Encaps	Decaps
128	702.4	715.9	701.0
192	1296.0	1397.3	1377.4
256	2591.3	2793.4	2819.2
384	5291.1	5554.6	5644.3
512	10714.8	11449.8	11475.8

C.3 SM3-based Results (Reference Implementation).

Table 25, Table 26, and Table 27 present the reference performance of Scloud⁺.KEM (SM3 family) on x86_64, Linux AArch64, and macOS arm64, respectively.

Table 25. The x86_64 reference performance of Scloud⁺.KEM with SM3, measured in 10³ cycles.

Level	KeyGen	Encaps	Decaps
128	22726.2	22790.8	22578.9
192	42267.9	42976.7	42910.2
256	89617.6	90748.4	89490.3
384	174577.4	174952.0	173878.7
512	361588.6	373921.7	372522.5

Table 26. The Linux AArch64 reference performance of Scloud⁺.KEM with SM3, measured in microseconds.

Level	KeyGen	Encaps	Decaps
128	8197.2	7991.5	7937.3
192	14912.4	15133.9	15001.0
256	30460.6	30907.6	30828.8
384	60833.8	61286.9	60709.1
512	127137.5	127847.1	127391.1

Table 27. The macOS arm64 reference performance of Scloud⁺.KEM with SM3, measured in microseconds.

Level	KeyGen	Encaps	Decaps
128	3154.5	3193.3	3166.8
192	5999.9	6157.8	6209.2
256	12726.9	12870.4	13014.3
384	24988.6	25551.3	25510.2
512	53605.1	53916.1	55289.9

References

- Anyu Wang, Zhongxiang Zheng, Chunhuan Zhao, Zhiyuan Qiu, Guang Zeng, Ye Yuan, Changchun Mu, and Xiaoyun Wang. Scloud⁺: An efficient LWE-based KEM without ring/module structure. In Xianhui Lu and Chris J. Mitchell, editors, *Security Standardisation Research - 9th International Conference, SSR 2024, Kunming, China, December 16, 2024, Proceedings*, Lecture Notes in Computer Science, pages 147–174. Springer, 2024.
- Zhongxiang Zheng, Anyu Wang, Haining Fan, Chunhuan Zhao, Chao Liu, and Xue Zhang. SCloud: Public key encryption and key encapsulation mechanism based on learning with errors. *IACR Cryptol. ePrint Arch.*, page 95, 2020.
- Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the Fujisaki-Okamoto transformation. pages 341–371, 2017.
- Michael Naehrig, Erdem Alkim, Joppe Bos, Léo Ducas, Karen Easterbrook, Brian LaMacchia, Patrick Longa, Ilya Mironov, Valeria Nikolaenko, Christopher Peikert, Ananth Raghunathan, and Douglas Stebila. FrodoKEM. Technical report, National Institute of Standards and Technology, 2020. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>.
- Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. pages 1–23, 2010.
- Chris Peikert, Oded Regev, and Noah Stephens-Davidowitz. Pseudorandomness of ring-LWE for any ring and modulus. pages 461–473, 2017.
- Adeline Langlois and Damien Stehlé. Worst-case to average-case reductions for module lattices. *Des. Codes Cryptogr.*, 75(3):565–599, 2015.

8. Peter Schwabe, Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Gregor Seiler, Damien Stehlé, and Jintai Ding. CRYSTALS-KYBER. Technical report, National Institute of Standards and Technology, 2022. available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>.
9. Jan-Pieter D’Anvers, Angshuman Karmakar, Sujoy Sinha Roy, Frederik Vercauteren, Jose Maria Bermudo Mera, Michiel Van Beirendonck, and Andrea Basso. SABER. Technical report, National Institute of Standards and Technology, 2020. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>.
10. Xianhui Lu, Yamin Liu, Dingding Jia, Haiyang Xue, Jingnan He, Zhenfei Zhang, Zhe Liu, Hao Yang, Bao Li, and Kunpeng Wang. LAC. Technical report, National Institute of Standards and Technology, 2019. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-2-submissions>.
11. Jiang Zhang, Yu Yu, Shuqin Fan, Zhenfeng Zhang, and Kang Yang. Tweaking the asymmetry of asymmetric-key cryptography on lattices: KEMs and signatures of smaller sizes. pages 37–65, 2020.
12. Ronald Cramer, Léo Ducas, Chris Peikert, and Oded Regev. Recovering short generators of principal ideals in cyclotomic rings. pages 559–585, 2016.
13. Ronald Cramer, Léo Ducas, and Benjamin Wesolowski. Short stickelberger class relations and application to ideal-SVP. pages 324–348, 2017.
14. Léo Ducas, Maxime Plançon, and Benjamin Wesolowski. On the shortness of vectors to be found by the ideal-SVP quantum algorithm. pages 322–351, 2019.
15. Ronald Cramer, Léo Ducas, and Benjamin Wesolowski. Mildly short vectors in cyclotomic ideal lattices in quantum polynomial time. *J. ACM*, 68(2):8:1–8:26, 2021.
16. ANSSI. <https://cyber.gouv.fr/en/publications/anssi-views-post-quantum-cryptography-transition>, 2022.
17. BSITechnical Guideline. Cryptographic mechanisms: Recommendations and key lengths, 2024.
18. Alex van Poppel. Cryptographic decoding of the leech lattice. Cryptology ePrint Archive, Report 2016/1050, 2016.
19. Shanxiang Lyu, Ling Liu, Junzuo Lai, Cong Ling, and Hao Chen. Lattice codes for lattice-based PKE. Cryptology ePrint Archive, Report 2022/874, 2022.
20. Vincent Corlay, Joseph J Boutros, Philippe Ciblat, and Loïc Brunel. On the decoding of Barnes-Wall lattices. In *2020 IEEE International Symposium on Information Theory (ISIT)*, pages 519–524. IEEE, 2020.
21. Daniele Micciancio and Antonio Nicolosi. Efficient bounded distance decoders for Barnes-Wall lattices. In *ISIT 2008, Toronto, ON, Canada, July 6-11, 2008*, pages 2484–2488. IEEE, 2008.
22. G. David Forney Jr. Coset codes-ii: Binary lattices and related codes. *IEEE Trans. Inf. Theory*, 34(5):1152–1187, 1988.
23. Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM (JACM)*, 56(6):1–40, 2009.
24. Joppe W. Bos, Craig Costello, Léo Ducas, Ilya Mironov, Michael Naehrig, Valeria Nikolaenko, Ananth Raghunathan, and Douglas Stebila. Frodo: Take off the ring! Practical, quantum-secure key exchange from LWE. pages 1006–1018, 2016.
25. Haodong Jiang, Zhenfeng Zhang, Long Chen, Hong Wang, and Zhi Ma. IND-CCA-secure key encapsulation mechanism in the quantum random oracle model, revisited. pages 96–125, 2018.
26. Haodong Jiang, Zhenfeng Zhang, and Zhi Ma. Tighter security proofs for generic key encapsulation mechanism in the quantum random oracle model. pages 227–248, 2019.
27. Claus-Peter Schnorr. A hierarchy of polynomial time lattice basis reduction algorithms. *Theor. Comput. Sci.*, 53:201–224, 1987.
28. Yuanmi Chen and Phong Q. Nguyen. BKZ 2.0: Better lattice security estimates. pages 1–20, 2011.
29. Yoshinori Aono, Yuntao Wang, Takuya Hayashi, and Tsuyoshi Takagi. Improved progressive BKZ algorithms and their precise cost estimation by sharp simulator. In *Advances in Cryptology–EUROCRYPT 2016*, pages 789–819. Springer, 2016.
30. Martin R. Albrecht, Shi Bai, Pierre-Alain Fouque, Paul Kirchner, Damien Stehlé, and Weiqiang Wen. Faster enumeration-based lattice reduction: Root hermite factor $\beta^{1/(2\beta)}$ in time $\beta^{\beta/8+o(\beta)}$. In *Advances in Cryptology–CRYPTO 2020*. Springer, 2020.
31. Martin R. Albrecht, Shi Bai, Jianwei Li, and Joe Rowell. Lattice reduction with approximate enumeration oracles: Practical algorithms and concrete performance. In *Advances in Cryptology–CRYPTO 2021, Part II*, pages 732–759. Springer, 2021.
32. Anja Becker, Léo Ducas, Nicolas Gama, and Thijs Laarhoven. New directions in nearest neighbor searching with applications to lattice sieving. pages 10–24, 2016.
33. Thijs Laarhoven. Search problems in cryptography: from fingerprinting to lattice sieving, 2016.

34. Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum key exchange - a new hope. In *25th USENIX Security Symposium (USENIX Security 16)*, pages 327–343, 2016.
35. Martin R. Albrecht, Florian Göpfert, Fernando Virdia, and Thomas Wunderer. Revisiting the expected cost of solving uSVP and applications to LWE. Cryptology ePrint Archive, Paper 2017/815, 2017.
36. André Chailloux and Johanna Loyer. Lattice sieving via quantum random walks. pages 63–91, 2021.
37. Martin R. Albrecht, Benjamin R. Curtis, Amit Deo, Alex Davidson, Rachel Player, Eamonn W. Postlethwaite, Fernando Virdia, and Thomas Wunderer. Estimate all the {LWE, NTRU} schemes! Cryptology ePrint Archive, Paper 2018/331, 2018.
38. MATZOV. Report on the security of LWE: Improved dual lattice attack, 2022.
39. Martin R. Albrecht, Benjamin R. Curtis, and Thomas Wunderer. Exploring trade-offs in batch bounded distance decoding. In *Selected Areas in Cryptography – SAC 2019*, pages 467–491. Springer, 2019.
40. Thomas Wunderer. A detailed analysis of the hybrid lattice-reduction and meet-in-the-middle attack. *Journal of Mathematical Cryptology*, 13(1):1–26, 2019.
41. Martin R Albrecht. On dual lattice attacks against small-secret LWE and parameter choices in HELib and SEAL. In *Advances in Cryptology EUROCRYPT 2017*, pages 103–129. Springer, 2017.
42. Thomas Espitau, Antoine Joux, and Natalia Kharchenko. On a dual/hybrid approach to small secret LWE: a dual/enumeration technique for learning with errors and application to security estimates of FHE schemes. In *Progress in Cryptology–INDOCRYPT 2020*, pages 440–462. Springer, 2020.
43. Qian Guo and Thomas Johansson. Faster dual lattice attacks for solving LWE with applications to CRYSTALS. pages 33–62, 2021.
44. Léo Ducas and Ludo N Pulles. Does the dual-sieve attack on learning with errors even work? In *Annual International Cryptology Conference*, pages 37–69. Springer, 2023.